Contents

1 O1 **3**
1.1 CoT 开山之作 . . . 3
1.2 from r to Q* . . . 4
1.3 自我奖励 . . . 4
1.4 self-play . . . 4
1.5 Cursor . . . 4
1.6 Let’s verify step by step . . . 4
1.7 O1 相关汇总 . . . 5
1.8 CoT 能让 Transformer 更强 . . . 5
1.8.1 CoT：打破 Transformer 的“并行诅咒” . . . 5
1.8.2 CoT 的理论基础：从电路复杂度到 Transformer 表达能力 . . . 5
1.8.3 CoT 的实验验证：从模加到电路值问题，CoT 全面胜出！ . . . 5
1.8.4 讨论 . . . 6
1.9 CoT or not . . . 6
1.10 test-time scaling . . . 6
1.11 inference scaling . . . 6
1.12 扩散模型的 inference scaling . . . 7
1.13 O1 的评估 . . . 7
1.14 LLM Reasoning . . . 7
1.15 开源模型 +O1 . . . 7
1.16 复现 O1 . . . 7
1.17 O1 Replication Journey . . . 7
1.18 ScoRe . . . 7
1.19 LeCo . . . 8
1.20 Marco-O1 . . . 8
1.21 OmniSearch . . . 8
1.22 Coconut . . . 8
1.23 O1 的安全机制 . . . 8
1.24 O1 架构 . . . 8
1.25 gemini 2.0 flash thinking . . . 8
1.26 小模型的 O1 . . . 9
1.27 meta-cot . . . 9
1.28 sky-T1 . . . 9
1.29 Monkey . . . 9
1.30 其他的一些讨论 . . . 9

2 DeepSeek R1 **9**
2.1 背景 . . . 10
2.2 DeepSeek-R1-Zero . . . 10
2.2.1 RL 算法 . . . 10
2.2.2 Reward modeling . . . 12
2.2.3 Training template . . . 13
2.2.4 效果 . . . 13
2.2.5 Self-evolution Process . . . 14
2.2.6 Aha Moment . . . 14
2.3 DeepSeek-R1 . . . 14
2.3.1 阶段 1：Cold Start . . . 14
2.3.2 阶段 2：Reasoning-oriented Reinforcement Learning . . . 15
2.3.3 阶段 3：Rejection Sampling and Supervised Fine-Tuning . . . 15
2.3.4 阶段 4：Reinforcement Learning for all Scenarios . . . 15
2.3.5 效果 . . . 15
2.4 蒸馏 . . . 16

2.4.1 效果 . . . 16
2.5 失败的尝试 . . . 16
2.5.1 PRM . . . 16
2.5.2 MCTS . . . 16
2.6 小结 . . . 17
2.7 使用注意 . . . 17
2.7.1 deepseek-r1 使用指南 . . . 17
2.7.2 openai 推理使用指南 . . . 18

3 R1 的部署 **19**

4 R1 的讨论 **19**

5 R1 前后的一些工作 **20**
5.1 S1 . . . 20
5.2 LIMO . . . 20
5.3 PRIME . . . 20
5.4 TPO . . . 20
5.5 Huginn . . . 21
5.6 Goedel-Prover . . . 21
5.7 Kimi K1.5 . . . 21
5.8 O1 Embedder . . . 21
5.9 bridge . . . 21
5.10 BFS-Prover . . . 21
5.11 CoE . . . 22
5.12 显式 CoT . . . 22
5.13 认知行为 . . . 22
5.14 DR, GRPO . . . 22
5.15 AReaL . . . 22
5.16 TAO . . . 22
5.17 deepcoder . . . 23
5.18 Think twice . . . 23
5.19 glm-z1 . . . 23
5.20 d1 . . . 23
5.21 regtool . . . 23
5.22 RLVR 的局限 . . . 24
5.23 RLFT . . . 24
5.24 小结 . . . 24

6 R1 的复现与部署 **24**
6.1 open-r1 . . . 24
6.2 unlock-deepseek . . . 24
6.3 open-reasoner-zero . . . 24
6.4 vllm 跑 gguf 的 r1 . . . 24
6.5 unsloth . . . 26
6.5.1 跑满血版 1.58bit infer . . . 26
6.5.2 让小模型有 reasoning 能力 . . . 26
6.6 Logic-RL 与 reinforce-lite . . . 26
6.6.1 reinforce-lite . . . 26
6.6.2 Logic-RL . . . 26

7 R1 的微调 **26**
7.1 ColossalAI . . . 26
7.1.1 lora sft 满血 deepseek V3/R1 . . . 26
7.1.2 用强化学习微调蒸馏版 DeepSeek . . . 27

8 OpenAI **27**

9 Qwen **27**
9.1 QwQ-max-preview . . . 27
9.2 QWQ-32B . . . 27

10 Claude **27**
10.1 Claude 3.7 Sonnet . . . 27

11 基于 llama 的推理模型 **28**
11.1 Llama Nemotron . . . 28

12 seed-thinking **28**
12.1 DAPO . . . 28
12.2 VAPO . . . 28

13 多模态推理 **28**
13.1 综述 . . . 28
13.2 视觉推理 . . . 28
13.3 R1V . . . 28
13.4 gemini 2.0 . . . 29
13.5 Video-T1 . . . 29
13.6 mureka o1(音乐) . . . 29
13.7 Kimi-VL . . . 29

14 统一的多模态模型 **29**
14.1 gemini 2.5 . . . 29
14.2 gemma-3 . . . 29
14.3 Qwen2.5-Omni . . . 30
14.4 Qwen3 . . . 30
14.4.1 预训练 . . . 30
14.4.2 后训练 . . . 30
14.4.3 使用 . . . 31

15 deep research **33**
15.1 local-deep-research . . . 33
15.2 CycleResearcher . . . 33

16 高效 reasoning **33**
16.1 reasoning economy . . . 33
16.2 stop overthinking . . . 34

17 deepseek **35**
17.1 GRM+SPCT . . . 35
17.1.1 RM 分类 . . . 35
17.1.2 SPCT . . . 35
17.2 DeepSeek-Prover-V2 . . . 36

18 推理模型总结 **37**

1 O1

1.1 CoT 开山之作

Chain of thought prompting elicits reasoning in large language models

1.2 from r to Q*

这就是 OpenAI 神秘的 Q*？斯坦福：语言模型就是 Q 函数

From r to Q*: Your Language Model is Secretly a Q-Function

在 DPO 的基础上，引入 LLM 里的 token-level 的 MDP，用二分类的 preference feedback。发现了 3 个点：

- 尽管 DPO 是作为上下文多臂赌博机而派生出来的，但 DPO 模型的隐含奖励可在每个 token 层面上进行解释。
- DPO 模型的 likelihood search 类似在 decoding 阶段寻找一个 reward function。即在 token 层面的阐述方式下，经典的基于搜索的算法（比如 MCTS）等价于在 DPO 策略上的基于似然的搜索。
- 初始策略和参考分布的选择对于确定训练期间隐性奖励的轨迹非常重要。

OpenAI 秘密武器「草莓」计划曝光！Q* 推理能力大爆发，逼近 AGI L2 里程碑

5 级路线图：

- L1：聊天机器人，具有对话能力的 AI。
- L2：推理者，像人类一样能够解决问题的 AI。
- L3：智能体，不仅能思考，还可以采取行动的 AI 系统。
- L4：创新者，能够协助发明创造的 AI。
- L5：组织者，可以完成组织工作的 AI。

Strawberry 模型的目的是为了使公司的 AI 不仅能生成查询答案，还能提前计划，足够自主且可靠地浏览互联网，进行 OpenAI 所称的「深度研究」。

类似Star: Self-taught reasoner bootstrapping reasoning with reasoning能够通过迭代创建自己的训练数据，来「自我提升」到更高的智能水平。

Nature 封面：AI 训练 AI，越训越离谱

AI models collapse when trained on recursively generated data，对应下载的pdf

1.3 自我奖励

Self-Rewarding Language Models

「用 AI 训 AI」这事靠谱吗？

1.4 self-play

清华、北大等发布 Self-Play 强化学习最新综述

OpenAI o1 强化学习背后的自博弈（Self-play）方法介绍

万字长文推演 OpenAI o1 self-play RL 技术路线

有想入坑 RL-LLM 的同学吗？这个开源项目一个 GPU 够了，完成后欢迎来月之暗面 ~

https://github.com/inspirai/TimeChamber

1.5 Cursor

Scaling Law 瓶颈，Cursor 编程为什么这么强？团队参与新研究掏出秘密武器

Planning In Natural Language Improves LLM Search For Code Generation

1.6 Let’s verify step by step

o1 基石论文火爆传阅，Ilya 仍是关键先生！核心项目清北校友闪光

Let’s Verify Step by Step

OpenAI 使用过程监督提升数学推理能力

https://openai.com/index/improving-mathematical-reasoning-with-process-supervision/

1.7 O1 相关汇总

(toread)

OpenAI o1 要跟，怎么跟？这个 GitHub 项目把解读、博客、相关论文一网打尽

刚刚，OpenAI 震撼发布 o1 大模型！强化学习突破 LLM 推理极限

张俊林：OpenAI o1 的价值意义及强化学习的 Scaling Law

北大对齐团队独家解读：OpenAI o1 开启「后训练」时代强化学习新范式

Noam Brown 早已预示 o1 强大推理能力，演讲深度解析 AI 推理研究脉络

OpenAI o1 模型的前世今生 -toread

1.8 CoT 能让 Transformer 更强

谷歌再次痛失好局！OpenAI o1 证实谷歌 ICLR 2024 论文价值「四位华人贡献」

Chain of Thought Empowers Transformers to Solve Inherently Serial Problems

1.8.1 CoT：打破 Transformer 的“并行诅咒”

传统的 Transformer 模型虽然在自然语言处理领域取得了巨大成功，但它有一个致命弱点：擅长并行计算，但不擅长串行推理。这就像一个超级聪明的孩子，能快速完成大量的计算题，但却无法理解简单的逻辑推理。

而 CoT (Chain of Thought，思维链) 技术的灵感来源于人类的思维过程，它可以让 Transformer 模拟人类的思考方式，通过生成一系列中间推理步骤，来解决那些需要逻辑推理的复杂问题。

1.8.2 CoT 的理论基础：从电路复杂度到 Transformer 表达能力

作者用电路复杂性理论来解释 CoT 的强大之处，将 Transformer 的计算过程与电路模型进行类比，并将 Transformer 能够解决的问题类别定义为“CoT 复杂性类”

他们证明了传统的 Transformer 模型（没有 CoT）只能解决 AC0 电路能够解决的问题，而 AC0 电路是一种计算能力非常有限的电路模型。但是，如果加入 CoT，Transformer 的表达能力将得到质的飞跃！作者用数学严格证明了：

只要 CoT 步骤足够多，Transformer 就能模拟任意大小的布尔电路，从而解决 P/poly 问题，这是一个包含了 P 问题的更大的问题类别，相当于证明了 CoT 可以让 Transformer 解决几乎所有可以用计算机解决的问题。

1.8.3 CoT 的实验验证：从模加到电路值问题，CoT 全面胜出！

为了进一步验证 CoT 的有效性，论文作者设计了四个核心问题：

- 模加：计算两个数的和，并对某个整数取模
- 排列组合：计算一组排列的组合
- 迭代平方：对一个数进行多次平方运算
- 电路值问题：计算一个布尔电路的输出值

其中，模加问题可以用并行计算高效地解决，而其他三个问题则需要串行计算。

实验结果表明：

- 对于模加问题，即使不使用 CoT，Transformer 也能取得不错的效果
- 但对于其他三个问题，使用 CoT 可以显著提高 Transformer 的准确率，尤其是在模型深度较浅的情况下

1.8.4 讨论

CoT 能让模型推理能力无上限？田渊栋、LeCun 下场反对：两层 MLP 还能模拟全世界呢

Transformer 推理天花板被谷歌打破？DeepMind 首席科学家亮出 84 页 PPT，却遭 LeCun 反对

1.9 CoT or not

o1 带火的 CoT 到底行不行？新论文引发了论战

To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning

1.10 test-time scaling

小模型越级挑战 14 倍参数大模型，谷歌开启 Test-Time 端新的 Scaling Law

Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

一句话：根据给定的 prompt 难度，动态地分配测试时（Test-Time）的计算资源。在预训练阶段花费更少的计算资源，而**在推理阶段花费更多，这种策略可能更好**。+ 针对（Process Reward Model, PRM，Let’s Verify Step by Step里提出的）进行搜索。+ PRM 可以在模型生成答案过程中的每个步骤都提供评分，用于引导搜索算法，动态调整搜索策略，通过在生成过程中识别错误或低效的路径，帮助避免在这些路径上浪费计算资源。+ 在测试时根据 prompt 自适应地更新模型的响应分布。+ 模型不是一次性生成最终答案，而是逐步修改和改进它之前生成的答案，按顺序进行修订（revision）。+ 并行采样：独立生成 N 个答案，+ 顺序修订：每个答案依赖于前一次生成的结果，逐步修订。

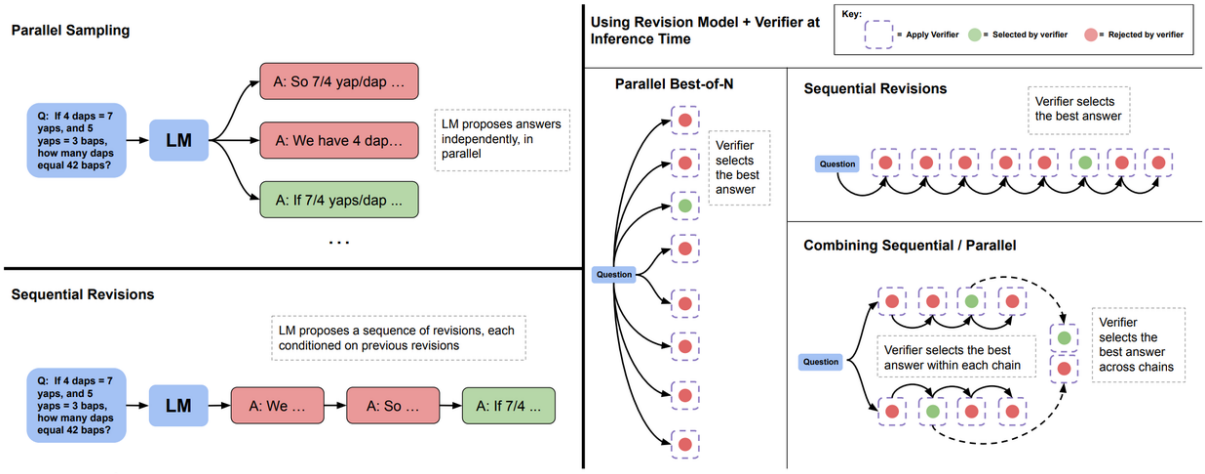


Figure 5 | ***Parallel sampling (e.g., Best-of-N) verses sequential revisions.*** **Left:** Parallel sampling generates N answers independently in parallel, whereas sequential revisions generates each one in sequence conditioned on previous attempts. **Right:** In both the sequential and parallel cases, we can use the verifier to determine the best-of-N answers (e.g. by applying best-of-N weighted). We can also allocate some of our budget to parallel and some to sequential, effectively enabling a combination of the two sampling strategies. In this case, we use the verifier to first select the best answer within each sequential chain and then select the best answer accross chains.

OpenAI o1 技术初探 1：整体框架，利用 Test-Time Scaling Law 提升逻辑推理能力—(写得比较细)

Scaling LLM Test-Time：谁说类 o1 推理一定要用 RL???

3B 模型长思考后击败 70B！HuggingFace 逆向出 o1 背后技术细节并开源

1.11 inference scaling

LLM Inference Scaling：姚班/OpenAI/CMU 8 月论文提前揭示 o1 核心原理

An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models

1.12 扩散模型的 inference scaling

扩散模型也能推理时 Scaling，谢赛宁团队重磅研究可能带来文生图新范式

Inference-Time Scaling for Diffusion Models beyond Scaling Denoising Steps

1.13 O1 的评估

280 页 PDF，全方位评估 OpenAI o1，Leetcode 刷题准确率竟这么高

Evaluation of OpenAI o1: Opportunities and Challenges of AGI

1.14 LLM Reasoning

一文看懂 LLM 推理，UCL 汪军教授解读 OpenAI o1 的相关方法

首个 o1 复现开源 RL 框架 OpenR 来了，UCL、上交等高校联合团队发布

OpenR: An Open Source Framework for Advanced Reasoning with Large Language Models

A Tutorial on LLM Reasoning: Relevant methods behind ChatGPT o1

大模型不会推理，为什么也能有思路？有人把原理搞明白了

Procedural Knowledge in Pretraining Drives Reasoning in Large Language Models

1.15 开源模型 +O1

OpenAI o1 式思维链，开源模型也可以有，成功案例来了

1.16 复现 O1

技术上，如何复现 o1?

17 岁高中生写了个神级 Prompt，直接把 Claude 强化成了满血 o1

https://github.com/richards199999/Thinking-Claude/tree/main

OpenAI 最大秘密，竟被中国研究者破解？复旦等惊人揭秘 o1 路线图

Scaling of Search and Learning: A Roadmap to Reproduce o1 from Reinforcement Learning Perspective

1.17 O1 Replication Journey

上交大发布首个 OpenAI o1 复现项目进展报告，满满的经验洞察

上交大 o1 复现新突破：蒸馏超越原版，警示 AI 研发”捷径陷阱”

O1 Replication Journey: A Strategic Progress Report

https://github.com/GAIR-NLP/O1-Journey/blob/main/resource/report-part2.pdf

https://github.com/GAIR-NLP/O1-Journey

1.18 ScoRe

强化学习让大模型自动纠错，数学、编程性能暴涨，DeepMind 新作

OpenAI o1 技术初探 3：如何让模型拥有自我纠错的能力

1.19 LeCo

COLM 24 | 从正确中学习？大模型的自我纠正新视角

Learning From Correctness Without Prompting Makes LLM Efficient Reasoner

https://github.com/starrYYxuan/LeCo

1.20 Marco-O1

阿里推理模型来了！Marco-o1 发布即开源

阿里国际版 o1 来了，Marco-o1：聚焦开放式问题推理

Marco-o1: Towards Open Reasoning Models for Open-Ended Solutions

https://github.com/AIDC-AI/Marco-o1

1.21 OmniSearch

阿里多模态检索智能体，自带 o1 式思考过程！复杂问题逐步拆解，动态调整下一步检索策略

Benchmarking Multimodal Retrieval Augmented Generation with Dynamic VQA Dataset and Self-adaptive Planning Agent

https://github.com/Alibaba-NLP/OmniSearch

1.22 Coconut

田渊栋团队论文火了！连续思维链优于 CoT，打开 LLM 推理新范式

(toread)

Training Large Language Models to Reason in a Continuous Latent Space

一般而言，LLM 被限制在语言空间（language space）内进行推理，并通过思维链（CoT）来表达推理过程，从而解决复杂的推理问题。然而，语言空间可能并不总是最适合推理的。例如，很多单词 token 主要用于文本连贯性，而不是推理本身，而一些关键 token 则需要复杂的规划。

Coconut（连续思维链，Chain of Continuous Thought）不再通过语言模型头（language model head）和嵌入层将隐藏状态与语言 token 进行映射，而是直接将最后的隐藏状态（即连续思维）作为下一个 token 的输入嵌入。

1.23 O1 的安全机制

OpenAI 发布 49 页长文，讲述 o1 的安全机制

OpenAI o1 System Card

1.24 O1 架构

「七万字长文」从认知架构到实践部署：o1 与 o1 Pro 的系统性分析与内涵洞察 · 上篇

1.25 gemini 2.0 flash thinking

推理最强也最快，谷歌发布 Gemini 2.0 Flash Thinking，全面超越 o1-preview

https://ai.google.dev/gemini-api/docs/thinking-mode?hl=zh-cn

https://colab.research.google.com/github/google-gemini/cookbook/blob/main/gemini-2/thinking.ipynb

我自己复制的一个：https://colab.research.google.com/github/daiwk/llms_new/blob/main/gemini-2/thinking.ipynb

对应的 github：https://github.com/daiwk/llms_new/blob/main/gemini-2/thinking.ipynb

1.26 小模型的 O1

(toread)

让 7B 千问模型超越 o1，微软 rStar-Math 惊艳登场，网友盛赞

rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking

https://github.com/microsoft/rStar

1.27 meta-cot

迈向 System 2 推理，100 页论文硬核讲述 Meta-CoT

Towards System 2 Reasoning in LLMs: Learning How to Think With Meta Chain-of-Thought

1.28 sky-T1

(toread)

450 美元训练一个「o1-preview」? UC 伯克利开源 32B 推理模型 Sky-T1，AI 社区沸腾了

https://novasky-ai.github.io/posts/sky-t1/

https://huggingface.co/NovaSky-AI/Sky-T1-32B-Preview

1.29 Monkey

o3 并非独门秘技，谷歌已发背后关键机制，方法更简单、成本更低

Large Language Monkeys: Scaling Inference Compute with Repeated Sampling

1.30 其他的一些讨论

OpenAI o1 模型超全指南来了！ (使用技巧)

耗资 1.3 万，ASU 团队揭秘 o1 推理王者！碾压所有 LLM 成本超高，关键还会 PUA

LLMs Still Can’t Plan; Can LRMs? A Preliminary Evaluation of OpenAI’s o1 on PlanBench

大模型是否有推理能力? DeepMind 数月前的论文让 AI 社区吵起来了

Amortized Planning with Large-Scale Transformers: A Case Study on Chess

联手 OpenAI，吴恩达推出一门 o1 推理新课程，还免费

https://www.deeplearning.ai/short-courses/reasoning-with-o1/

4o-mini 只有 8B，o1 也才 300B！微软论文意外曝光 GPT 核心机密

- o1-preview 约 300B；o1-mini 约 100B
- GPT-4o 约 200B；GPT-4o-mini 约 8B
- Claude 3.5 Sonnet 2024-10-22 版本约 175B

2 DeepSeek R1

DeepSeek-R1 发布，性能对标 OpenAI o1 正式版

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning，自己转存了一份：pdf

arxiv 上：https://arxiv.org/pdf/2501.12948

https://huggingface.co/deepseek-ai/DeepSeek-R1，还有不少 distill 的，LM-studio 已经有了。。https://hf-mirror.com/lmstudio-community/DeepSeek-R1-Distill-Qwen-32B-GGUF

Jay Alammar：图解 DeepSeek-R1

https://newsletter.languagemodels.co/p/the-illustrated-deepseek-r1

https://levelup.gitconnected.com/drawing-deepseek-r1-architecture-and-training-process-from-scratch-72043da33955

2.1 背景

o1 提出了可以通过增加 cot reasoning process 的长度来进行 inference time scaling，尝试复现的方法有：

- process-based reward models：
  - Let' s Verify Step by Step
  - Solving math word problems with process-and outcome-based feedback
  - Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations
- reinforcement learning：
  - Training language models to self-correct via reinforcement learning
- MCTS/beam search 的搜索方法：
  - Alphazero-like tree-search can guide large language model decoding and training
  - Solving olympiad geometry without human demonstrations
  - Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search

但这些方法都没有 o1 效果好，因此 R1 不借助监督数据，使用纯 RL(没有 SFT) 来完成 self-evolution，从而探索 LLM 在 reasoning 上的潜能。使用 DeepSeekV3 作为 base model，并使用Deepseekmath: Pushing the limits of mathematical reasoning in open language models的 GRPO 作为 RL 框架。

几 k 个 step 后，DeepSeek-R1-Zero 在一些 reasoning 的 benchmark 上取得了不错的效果，但仍然有可读性差、语言混合等问题，因此搞了 DeepSeek-R1，包括少量的冷启数据和一个多阶段的训练 pipeline。

- 收集数千的 cold-start 数据来 finetune DeepSeek-V3-Base 模型
- 执行类似 DeepSeek-R1-Zero 的面向 reasoning 的 RL 任务
- 在 RL 快收敛的时候，在 RL 的 checkpoint 上通过拒绝采样构建新的 SFT 数据，并和 DeepSeek-V3 的数据集（只选一些领域，如 writing、factual QA 和 self-cognition, 自我认知）进行结合，**重训** DeepSeek-V3-Base
- 用新数据 finetune 完后，再经过一个 RL 的过程，并考虑所有场景的 prompts==> 最终的模型

还搞了一些蒸馏模型，例如 Qwen2.5-32B 作为 base 模型，蒸馏 DeepSeek-R1 的效果比对它进行 RL 效果要好，说明大的 base model 发现的 reasoning patterns 对提升 reasoning 能力很关键。此外，发现 14B 的蒸馏模型比 QWQ-32B-preview 效果好很多，32B 和 70B 的蒸馏模型效果更好

2.2 DeepSeek-R1-Zero

2.2.1 RL 算法

GRPO（Group Relative Policy Optimization）：放弃了 Critic model，用 group scores 来替换。即对每个问题 q，从老的策略 $\pi_{\theta_{\text{old}}}$ 采样出一个 group 的输出 $\{o_1, o_2, \cdots, o_G\}$，再通过最大化如下目标来优化 policy model π_θ：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O \mid q)\right]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{old}}(o_i \mid q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{old}}(o_i \mid q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta || \pi_{ref}\right)\right),$$

其中：

- $\mathbb{D}_{KL}\left(\pi_\theta \| \pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1$ 用于限制新策略不要离 ref model 太远

- clip $\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)$ 用于限制新旧策略的概率比例在 1 附近，即二者不要偏离太远
- ε 和 β 是超参
- A_i 是 advantage，通过每个 group 的 reward 计算得来，不像 ppo 需要借助 value model：

$$A_i = \frac{r_i - \text{mean}\left(\{r_1, r_2, \cdots, r_G\}\right)}{\text{std}\left(\{r_1, r_2, \cdots, r_G\}\right)}$$

整句话的 advantage 如何分配到 token 上？参考 deepseekmath 论文：各 token 共享

4.1.2. Outcome Supervision RL with GRPO

Formally, for each question q, a group of outputs $\{o_1, o_2, \cdots, o_G\}$ are sampled from the old policy model $\pi_{\theta_{old}}$. A reward model is then used to score the outputs, yielding G rewards $\mathbf{r} = \{r_1, r_2, \cdots, r_G\}$ correspondingly. Subsequently, these rewards are normalized by subtracting the group average and dividing by the group standard deviation. Outcome supervision provides the normalized reward at the end of each output o_i and sets the advantages $\hat{A}_{i,t}$ of all tokens in the output as the normalized reward, i.e., $\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$, and then optimizes the policy by maximizing the objective defined in equation (3).

- 大基座是出现 reasoning 涌现能力的关键（qwen 32bb 用同样 r1-zero 的数据训练，没法出现 aha moment）
- GRPO 可并行，可加速

对比 PPO & GRPO

- PPO：
  - Reference Model（Ref model）：原始 LLM，不训练
  - Policy Model（Actor）：原始 LLM，需要训练
    - policy：LLM 产出的一条 response；
    - action：LLM 生成的每一个 token
  - Reward Model（RM）：对一条 response 的打分。在 RLHF 的 RM 阶段更新，RL 阶段不训练
  - Value Model（Critic）：类似 RM，需要训练
  - GAE：从 Reward Model 对**整个句子**的打分，拆解到**每个 token** 应该怎么改进（图中的 A，advantage），会考虑 Value Model、Ref Model 和 Reward Model，还会有一个 KL 的约束，确保新的 policy model 不会离 ref model 太远
  - Importance Sampling：对比新 advantage 和原始 policy 的比值，只对 advantage 进行小比例更新，保证训练稳定
  - 缺点：
    - 需要 Ref Model 再算一遍，有计算量 & 耗时
    - 需要拆解到每个 token，又有计算量 & 耗时
- GRPO：（Group Relative Policy Optimization）：来自DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models，放弃了 Critic model（图中的 value model），用 group scores 来替换。
  - policy model/ref model/reward model 都和 ppo 一样
  - 不需要 value model
  - 并行执行更多的 policy（即 GRPO 的 group），产出的结果一起打分，减均值除以标准差，把每个 policy 和标准差的差距平均到每个 token 上，再用 KL 散度计算和 ref model 的差距

原始 DeepSeekMath 论文中的图：

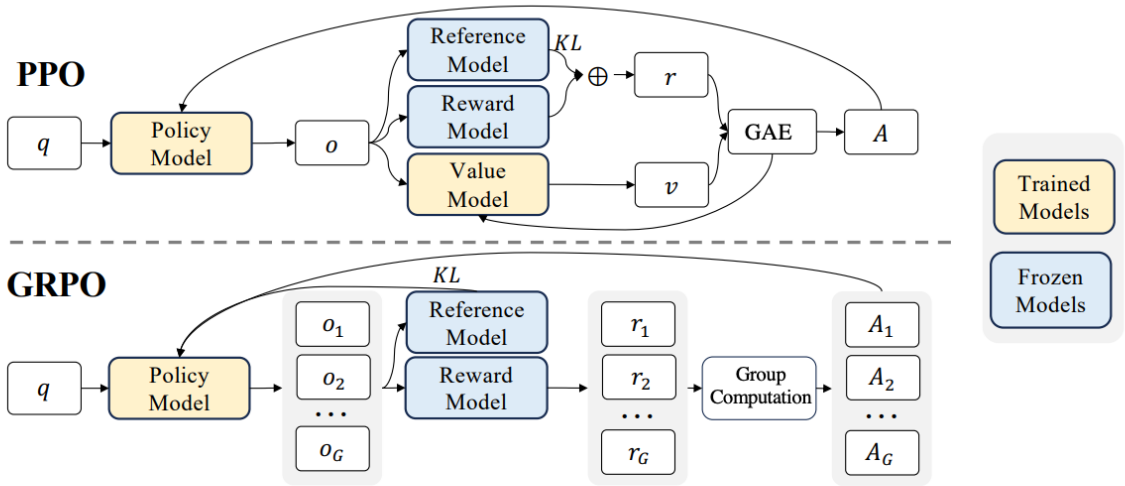
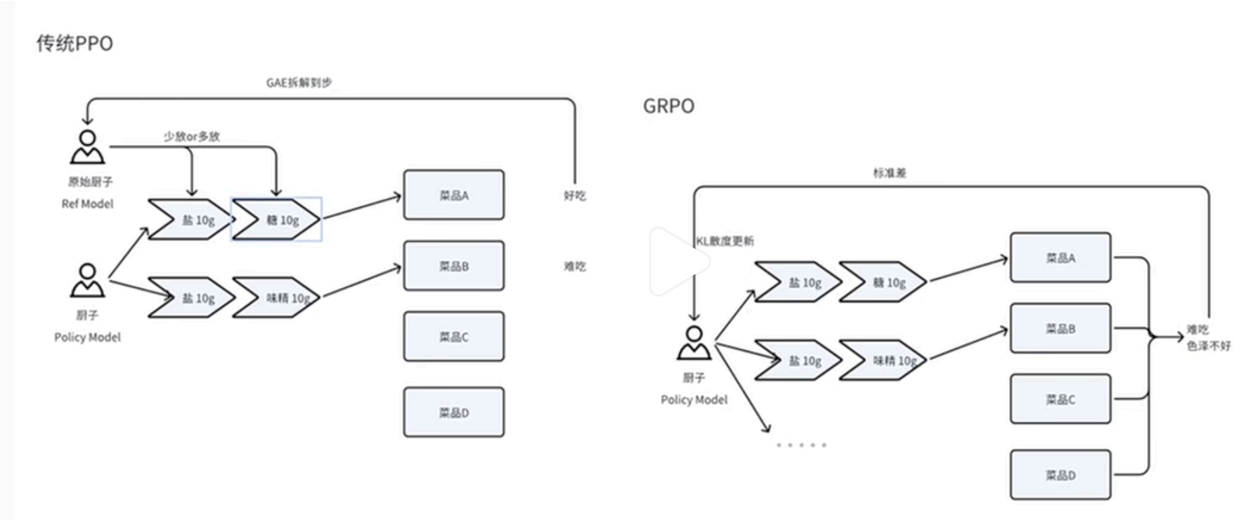


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

形象的理解：



从头开始实现：DeepSeek 关键 RL 算法 GRPO，有人从头跑通了，贡献完整代码

2.2.2 Reward modeling

用的是 rule-based reward，包括如下两部分

- accuracy rewards：response 是否正确，例如对于一个数学问题，需要判断最终答案是否为给定的格式（specified format），并且能基于规则验证结果的正确性；leetcode 问题则可以用编译器来生成预先定好的测试用例的结果。
- format rewards：设计了一个 format reward model，来强制模型将其思考过程输出在 `<think>` 和 `</think>` 之间。

没有设计 neural reward model，因为：

- 发现在大规模的 RL 过程中，neural reward model 会受 reward hacking 的影响
- 重新训练 reward model 需要额外的训练资源，并且会让训练 pipeline 过于复杂

2.2.3 Training template

```
A conversation between User and Assistant. The user asks a question, and the Assistant solves it.
The assistant first thinks about the reasoning process in the mind and then provides the user
with the answer. The reasoning process and answer are enclosed within <think> </think> and
<answer> </answer> tags, respectively, i.e., <think> reasoning process here </think>
<answer> answer here </answer>. User: prompt. Assistant:
```

训练时使用如上的 template，故意将约束限制在这种结构格式上，避免任何特定内容的偏见（例如强制反射推理或促进特定的解决问题的策略），以确保能够在 RL 过程中准确观察模型的自然进展。

2.2.4 效果

指标含义：

- pass@1：**首次**生成答案时的成功率
- cons@64：majority vote (consensus) with 64 samples，模型在给出 **64 次生成尝试**中，是否能够**多次生成相同的正确答案**。(在这里提到了，来自文章：Self-consistency improves chain of thought reasoning in language model)

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

Table 2 | Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

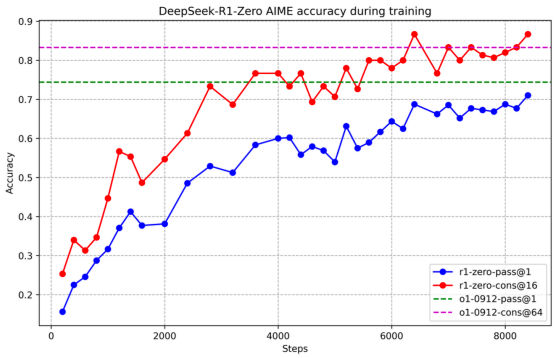


Figure 2 | AIME accuracy of DeepSeek-R1-Zero during training. For each question, we sample 16 responses and calculate the overall average accuracy to ensure a stable evaluation.

- 数学 (美国数学邀请赛)：
  - AIME2024：pass@1 上接近 o1-0912，cons@64 超了，均超越 o1-mini
  - MATH-500：pass@1 超越 o1-mini 和 o1-0912
- GPQA-Diamond（phd 级别的科学问题，包括化学、物理、生物）：pass@1 超越 o1-mini，不如 o1-0912
- 代码：
  - LiveCode Bench：不如 o1-0912 和 o1-mini
  - codeforces 的 rating：不如 o1-0912 和 o1-mini

另外，在 AIME2024 上，随着训练，pass@1 能从十几涨到 70 几。

2.2.5 Self-evolution Process

- 随着训练的进行，生成的 response 长度也在变长，说明思考时间也变长了，其中的 reasoning tokens 从数百涨到了数千。
- 随着测试时间的增加，**涌现出了复杂的行为**，即不由自主地 (spontaneously) 出现了（这些行为并非预先在代码里设计的）：
  - 反思：重新看并且评估 (revist and reevaluate) 之前的 steps
  - 探索：尝试其他方法来解决问题

2.2.6 Aha Moment

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a-\sqrt{a+x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a-\sqrt{a+x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a-\sqrt{a+x}}\right)^2 = x^2 \implies a-\sqrt{a+x} = x^2$. Rearrange to isolate the inner square root term: $(a-x^2)^2 = a+x \implies a^2-2ax^2+(x^2)^2 = a+x \implies x^4-2ax^2-x+(a^2-a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a-\sqrt{a+x}} = x$ First, let's square both sides: $a-\sqrt{a+x} = x^2 \implies \sqrt{a+x} = a-x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

在某一个中间版本，模型突然输出上面的 aha-moment，可以看到不需要显式地教模型如何解决，只要给予正确的激励（incentives），模型会自己开发出进阶的问题解决策略，这也是 RL 的 power and beauty。

2.3 DeepSeek-R1

2 个目标：

- 用一小部分高质量数据来做冷启能否提升推理效果，或者加速收敛？
- 如何训练一个 user-friendly 的模型，既能提供准确和连贯的 CoT，又有很强的通用能力？

2.3.1 阶段 1：Cold Start

从 base model 开始 RL 训练时，为了防止冷启阶段训练的不稳定，构建了少量（数千）的长 CoT 数据来 finetune 模型，作为初始的 RL actor。试了几种方法

- few shot prompting，用一个 long CoT 作为例子
- 直接在 prompt 里要求模型生成详细的答案，并加上反思和验证
- 收集 deepseek-r1-zero 的输出，并找人类标注员后处理进行修改

相比 zero，冷启数据有如下好处：

- 可读性：zero 的可读性不太好，例如语言混合、缺少 markdown 高亮。因此设计了一个可读性比较好的 pattern，即 `|special_token|<reasoning_process>|special_token|<summary>`，并把没有 summary 的结果扔掉
- 潜力：依据人类先验精心设计的 pattern 产出的冷启数据，取得了比 zero 更好的效果，故迭代地训练对于 reasoning models 应该是一种更好的方式

2.3.2 阶段 2：Reasoning-oriented Reinforcement Learning

在冷启数据上 tune 完之后，和 r1-zero 一样，进行大规模的 RL 训练（在 reasoning-intensive 任务的数据集上训的?）。此外，为了解决 CoT 里语言混合的问题，加了一个 **language consistency reward**（计算目标语言在 CoT 中的比例），直接和原来的 reward 相加。虽然消融实验显示这种 alignment 会让效果略微下降，但这更符合人类偏好，可读性更强。

2.3.3 阶段 3：Rejection Sampling and Supervised Fine-Tuning

用阶段 2 的 ckpt，然后在 writing, role-playing, and other general-purpose 这类任务上构建 SFT 数据集，在约 80w 的样本上对 DeepSeek-V3-Base SFT 了 2 个 epoch。

- reasoning data：在前一个 RL 阶段，只用那些能用 rule-based rewards 来衡量的数据。
  - 这个阶段加入了更多的数据，有一些是使用**生成式的 reward model**，即把 ground truth 和模型预测结果输入给 DeepSeek-V3 做 judgement。
  - 因为模型输出有时会很混乱或者无可读性，所以删掉一些满足这些条件的 CoT：混合语言、长的段落、代码段
  - 对每个 prompt，采样多个 response，并只保留一个正确的。最终收集了大概 60w 的 reasoning 相关的训练样本
- non-reasoning data：
  - 采用 DeepSeek-V3 的 pipeline，并复用部分 DeepSeek-V3 的 SFT 数据集。
  - 使用 DeepSeek-V3 时，在 prompt 里提到在回答答案前要生成潜在的 CoT；对于简单的如 hello 的问题，就不用输出 CoT。
  - 最终收集了大概 20w 的 non-reasoning 训练样本

2.3.4 阶段 4：Reinforcement Learning for all Scenarios

将 reward 信号和多样化的数据分布相结合，提升 helpfulness and harmlessness，同时 refine reasoning 能力，

- reasoning 数据：用 R1-Zero 的 RL，使用 rule-based reward
- 通用数据：用 V3 的 pipeline，还是用 reward model，用以学习复杂和细分领域的人类偏好

另外，提升如下两方面的能力：

- helpfulness：仅关注最终的 summary，保证评估强调的是 response 的有用性和相关性，而尽量少地干涉潜在的 reasoning 过程
- harmlessness：同时评估 reasoning 过程和 summary，尽量缓解在生成的过程中出现的风险、偏见和有害内容

2.3.5 效果

- 大部分都接近或者超过新版 o1（r1-zero 只能和早期 o1 相当），很多任务都比 v3 要好
- 代码和 o1 差不太多，比 v3 提升很多，其中 SWE Verified 这个工程类的不太行，因为目前训练语料里这类数据还比较少，下一版会优化
- c-simpleQA（最后一行）上效果不如 V3，因为有 R1 有 safety 的限制，导致模型会拒绝回答一些问题，如果去掉这个限制，准确率能达到 70%
- 在 AlpacaEval2.0（写作）和 ArenaHard(开放领域问答) 上效果很好，产出的 summary 也很长，说明可能 reasoning 能力能产出长 cot，对这类任务也有用
- 数学上超越 o1，大幅领先 v3

2.4 蒸馏

基于 Qwen2.5-Math-1.5B、Qwen2.5-Math-7B、Qwen2.5-14B、Qwen2.5-32B、Llama-3.1-8B、Llama-3.3-70B-Instruct 蒸馏，用的数据集是 deepseek-R1 的第二阶段 RL 使用的 80w 数据，只进行了 SFT，没有 RL。理论上 RL 应该会效果更好，留给其他研究者去搞了。

https://medium.com/@prabhudev.guntur/how-to-distill-deepseek-r1-a-comprehensive-guide-c8ba04e2c28c可以参考这个自己蒸馏一个

2.4.1 效果

- 仅 sft 蒸馏的效果：模型越大效果越好，32b 和 70b 的已经在大部分任务上超越 o1-mini 了
- 直接拿 qwen-32b 做 RL（第二行）：和 QwQ-32b-preview（第一行）差不太多，但明显不如 sft 蒸馏（第 3 行）的模型效果

最终结论：

- 把大模型蒸馏到小模型可以有很好的效果，但对小模型做 RL 性价比不高
- 有更强大的模型和更大规模的 RL，才能更好地发挥蒸馏的作用，让小模型变得更强

2.5 失败的尝试

2.5.1 PRM

PRM（Process-supervised Reward Model）是 OpenAI 在Let' s Verify Step by Step中首次提出的概念。与之相对应的是 ORM（Outcome-supervised Reward Model）。区别：

- PRM：过程奖励模型，在生成过程中，对每一个**步骤**打分，是更细粒度的奖励模型。
- ORM：结果奖励模型，不管推理有多少步，对**完整的生成结果**进行**一次**打分，是一个反馈更稀疏的奖励模型。

PRM 可以在两个阶段生效：

- Post-Training 阶段：在 RL 过程中增加 PRM，对采样的结果**按步骤**输出奖励值，为模型提供更精细的监督信号，来指导策略模型优化，提升模型按步推理的能力。
- Inference 阶段：对 generator 模型做 N 次采样（如 Beam Search 等），并通过 PRM 对每个采样的每步推理进行打分，最终拟合一个整体过程打分，并选取打分最高的结果作为最终的答案。

问题：

- 难以定义细粒度的步骤
- 评估中间步骤是否正确很困难，用 LLM 自动评估结果不太行，手动标注又难以规模化应用
- 基于模型的 PRM 就会有 reward hacking 问题，重新训练奖励模型需要额外的训练资源

2.5.2 MCTS

MCTS（Monte Carlo Tree Search）是强化学习领域，详见 AlphaGo 论文Mastering the game of Go without human knowledge

具体操作步骤：使用已有的策略与环境做仿真交互，进行多次 rollout 采样，最终构成了一个从当前节点出发的一颗 Tree（每个 rollout 表示从当前节点到最终结束状态的多次与环境仿真交互的过程）。

这棵树的所有叶子节点都是结束状态，结束状态是可以量化收益的（比如方法 1：答案错误收益-1，答案正确收益 +3；再比如方法 2：叶子节点的收益是到达叶子节点路径数/总路径数的概率，这是一种根据投票机制预估的价值，越多路径到达叶子节点，说明这个叶子节点越置信，那么这个叶子节点就有更高的奖励）。

一棵树的叶子节点有了奖励值，就可通过反向传播，计算每个中间节点的奖励值，最终计算出整个树所有节点的奖励值。MCTS 一次 rollout 包括四个步骤：

- select
- expand

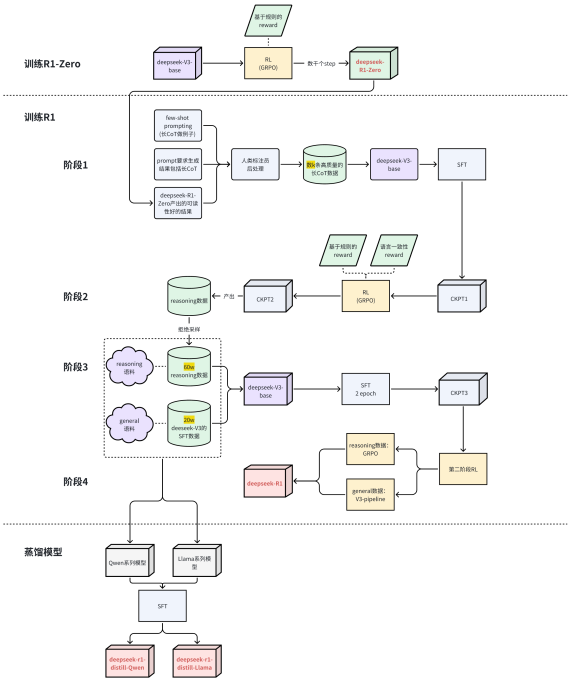
- simulate
- backprop

MCTS 也可以用在两阶段：

- Post-Traing 阶段：对于**每个 problem 构造一个搜索树**，然后进行树的游走遍历采样，再用采样的样本 SFT 或 RL 训练模型。
- Inference 阶段：在推理阶段，也是对一个 problem 探索多节点构造一棵搜索树，对于到达正确答案的路径，根据节点路径的置信度打分，**贪心选取最优路径**作为最终的推理结果。

问题：＋搜索空间巨大，虽然设置最大扩展限制使得不会无限搜索，但是容易陷入局部最优＋value model 直接影响生成的质量，而训练一个细粒度的 value model 本质上是困难的

2.6 小结



对应的流程图文档

2.7 使用注意

2.7.1 deepseek-r1 使用指南

刚刚，DeepSeek 官方发布 R1 模型推荐设置，这才是正确用法

- temperature 设置在 0.5-0.7 之间，推荐 0.6，以防止无休止的重复或不连贯的输出。
- 不要 system prompt，直接放到 user prompt 里
- 数学问题，建议 prompt 里直接 `Please reason step by step, and put your final answer within \boxed{}.`
- 有些时候模型会自己跳过 think 的过程，如果要强制输出，可以强制模型在 output 前输出 `<think>\n`

上传文件的 prompt：

```
file_template = \
"""[file name]: {file_name}
[file content begin]
{file_content}
```

```
[file content end]
{question}"""
```

联网的 prompt:

```
search_answer_zh_template = \
'''# 以下内容是基于用户发送的消息的搜索结果:
{search_results}
在我给你的搜索结果中，每个结果都是 [webpage X begin]...[webpage X end] 格式的，X 代表每篇文章的数字索引。请在适当的情况下在句子末
在回答时，请注意以下几点:
- 今天是 {cur_date}。
- 并非搜索结果的所有内容都与用户的问题密切相关，你需要结合问题，对搜索结果进行甄别、筛选。
- 对于列举类的问题（如列举所有航班信息），尽量将答案控制在 10 个要点以内，并告诉用户可以查看搜索来源、获得完整信息。优先提供信息完整、最相关的
- 对于创作类的问题（如写论文），请务必在正文的段落中引用对应的参考编号，例如 [citation:3][citation:5]，不能只在文章末尾引用。你需要解读
- 如果回答很长，请尽量结构化、分段落总结。如果需要分点作答，尽量控制在 5 个点以内，并合并相关的内容。
- 对于客观类的问答，如果问题的答案非常简短，可以适当补充一到两句相关信息，以丰富内容。
- 你需要根据用户要求和回答内容选择合适、美观的回答格式，确保可读性强。
- 你的回答应该综合多个相关网页来回答，不能重复引用一个网页。
- 除非用户要求，否则你回答的语言需要和用户提问的语言保持一致。

# 用户消息为:
{question}'''
```

2.7.2 openai 推理使用指南

刚刚，DeepSeek 揭秘 R1 官方同款部署设置，温度 =0.6！OpenAI 推理指南同时上线

https://platform.openai.com/docs/guides/reasoning-best-practices

简单理解:

- 推理模型就像一位经验丰富的高级同事——你只需告诉他们最终目标，就能相信他们自主完成所有细节工作。
- GPT 模型则更像一位新手同事——你需要提供明确详细的指示，才能让他们准确完成特定的输出任务。

各自特点:

- 速度和成本：选择 GPT 模型，因为它们处理速度更快，成本更低
- 执行明确任务：选择 GPT 模型，它们在处理界定清晰的任务时表现出色
- 准确性和可靠性：选择 o 系列模型，它们是可靠的决策专家
- 复杂问题解决：选择 o 系列模型，它们善于处理模糊和复杂的问题

推理模型的优势:

- 处理模糊任务：推理模型特别擅长处理信息有限或零散的情况，只需通过简单的提示词就能理解用户意图并妥善处理指令中的信息缺口。值得注意的是，推理模型通常会在做出未经验证的猜测或填补信息空缺之前，主动提出澄清性问题。
- 大海捞针：当需要处理大量非结构化信息时，推理模型特别擅长理解内容并精准提取出回答问题所需的关键信息。
- 在大型数据集中发现关系和细微差别:
  - 推理模型特别擅长分析包含数百页密集、非结构化信息的复杂文档，如法律合同、财务报表和保险索赔等。这些模型在识别文档之间的关联性，并基于数据中隐含的事实做出决策方面，表现尤为突出。
  - 推理模型还特别擅长理解细微的政策和规则，并将其准确应用于具体任务中以得出合理结论。
- 多步骤 AI 智能体规划：推理模型在 AI 智能体规划和策略制定中发挥着关键作用。将推理模型作为「计划者」时效果显著：它能为问题制定详细的多步骤解决方案，并根据具体需求（高智能或低延迟）选择和分配合适的 GPT 模型（执行者）来完成各个步骤。
- 视觉推理能力：截至目前，o1 是唯一一个具备视觉处理能力的推理模型。与 GPT-4o 相比，o1 的独特优势在于它能够准确理解最具挑战性的视觉内容，包括结构不规则的图表和表格，以及质量欠佳的图片。
- 代码审查、调试和质量改进：推理模型在审查和改进大规模代码方面表现突出。考虑到这类模型的较高延迟特性，通常将代码审查任务安排在后台运行。虽然 GPT-4o 和 GPT-4o mini 凭借较低的延迟可能更适合直接编写代码，但在那些对延迟要求相对不那么严格的代码生成场景中，o3-mini 表现同样出色。

- 评估和基准测试其他模型的响应：推理模型在对其他模型的输出进行基准测试和评估方面表现优异。数据验证对确保数据集的质量和可靠性至关重要，这一点在医疗保健等敏感领域尤其重要。

prompt 注意点：

- 用开发者消息取代系统消息：自 o1-2024-12-17 版本起，推理模型开始支持开发者消息（developer message）而非系统消息（system message）。
- 保持提示词简洁明确：推理模型最擅长理解和响应简短、清晰的指令。
- 避免使用 CoT 提示：由于模型内置推理能力，因此无需特别提示它们「一步一步思考」或「解释推理过程」。
- 善用分隔符增强清晰度：使用 Markdown、XML 标签和章节标题等分隔符来明确区分输入的不同部分，这有助于模型准确理解各个章节的内容。
- 优先尝试零样本学习：推理模型通常无需少样本示例即可产出优质结果，因此建议先尝试不含示例的提示词。如果对输出结果有更复杂的要求，再考虑在提示词中添加输入和期望输出的示例。请注意确保示例与提示词指令严格匹配，因为不一致可能导致性能下降。
- 提供明确约束条件：如果需要对模型的响应施加具体限制（例如「提供预算控制在 500 美元以内的解决方案」），请在提示词中明确列出这些约束条件。
- 明确定义目标：在指令中，请详细说明判定响应成功的具体参数，并引导模型持续优化推理过程，直到达成设定的成功标准。
- Markdown 格式说明：从 o1-2024-12-17 版本开始，API 中的推理模型默认不会生成带有 Markdown 格式的响应。如果确实需要在响应中包含 Markdown 格式，请在开发者消息的首行添加「Formatting re-enabled」字符串。

3 R1 的部署

完整的 671B MoE DeepSeek R1 怎么塞进本地化部署？详尽教程大放送！

https://github.com/kvcache-ai/ktransformers

4 R1 的讨论

华人研究团队揭秘：DeepSeek-R1-Zero 或许并不存在「顿悟时刻」

为什么说 DeepSeek 的 R1-Zero 比 R1 更值得关注？

LeCun 痛批硅谷傲慢病！圈内爆火长文：DeepSeek R1-Zero 比 R1 更重要，成 AGI 破局关键

DeepSeek 用的 GRPO 占用大量内存？有人给出了些破解方法，对应：https://github.com/huggingface/trl/issues/2709

刘知远硬核解读 DeepSeek：大模型强化学习技术原理与大模型技术发展研判

大神卡帕西拿 DeepSeek R1 讲强化学习！最新大模型内部机制视频爆火，“没有技术背景也能看懂”

对应代码：https://github.com/EurekaLabsAI

陈巍：DeepSeek 是否有国运级的创新？2 万字解读与硬核分析 DeepSeek V3/R1 的架构

Sebastian Raschka：关于 DeepSeek R1 和推理模型，我有几点看法

4500 美元复刻 DeepSeek 神话，1.5B 战胜 o1-preview 只用 RL！训练细节全公开

从想太多到想不透？DeepSeek-R1 等长推理模型也存在「思考不足」问题

r1 技术分享

一些观点：

- 打破 LLM 训练从 Generalist 到 Reasoner 的常规思路，使用 RL 先得到 Reasoner，再经过 SFT 成为 Generalist;
- GRPO 算法的高效性，让 RL 大规模训练取得了效果;
- SFT with CoT 数据的训练结果在原文中并未给出，但从最近一些复现工作和 R1 distiled Qwen 结果来看，SFT 的作用可能比 RL 还大。
- 好的 Base 模型本身就有 Aha Moment，但是是 Superfacial（表面）的 reflection，不具有提升 Accuracy 的能力，RL 可以增强该能力。

r1 的解读（很长，142 页）

DeepSeek-R1 Thoughtology: Let’s about LLM reasoning

模型思考过程分四步走：

- 问题定义：先把问题说清楚
- 开花期：拆解问题，给出初步方案
- 重构期：反复验证和修正想法
- 最终决策：确认并输出答案

有趣的发现：

- 思考链长度有”最佳区间”，太长反而会答错
- 模型会”死磕”用户给的信息，即使信息是错的也坚持跟着走
- 中英文环境下性格迥异：中文更重视集体，英文更个人主义
- 还会画 ASCII 艺术，虽然不太完美 ~

缺陷：

- 上下文太长容易”走神”
- 有时会生成有害内容
- 在某些视觉任务上还不够连贯

5 R1 前后的一些工作

5.1 S1

s1: Simple test-time scaling

https://github.com/simplescaling/s1

训练 1000 样本就能超越 o1，李飞飞等人画出 AI 扩展新曲线

（从 xhs 上抄的）有几点思维误区：

- s1 超过的是 o1-preview，没有超过 o1 甚至 o1-mini，有很大区别
- s1 的效果不敌 deepseek-r1 800k 数据蒸馏的 32B 模型，差了不少，不是媲美
- s1 即使使用全量 59k 数据的效果也没有提高很多，甚至在 math 上还有下降，所以核心是数据质量
- 1k 数据是从 59K 数据中筛选出来的，不是直接有 1K 数据就可以
- s1 使用 1k 数据是节省了训练时间，但蒸馏的难点在蒸馏数据的构造上

5.2 LIMO

817 样本激发 7 倍推理性能：上交大「少即是多」定律挑战 RL Scaling 范式

LIMO: Less is More for Reasoning

https://github.com/GAIR-NLP/LIMO

5.3 PRIME

挑战 DeepSeek-R1-Zero！PRIME：仅 10% 数据刷新数学推理 SOTA，隐式奖励技术颠覆 RL 训练

Process Reinforcement through Implicit Rewards

5.4 TPO

推理时也能做偏好优化，无需额外重训练，来自上海 AI Lab 港中文等

Test-Time Preference Optimization: On-the-Fly Alignment via Iterative Textual Feedback

https://github.com/yafuly/TPO

5.5 Huginn

推理模型新路线开源！与 DeepSeek 截然不同，抛弃思维链不用人类语言思考

Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach

https://github.com/seal-rg/recurrent-pretraining

抛弃长思维链和人类的语言，直接在连续的高维潜空间用隐藏状态推理，可自适应地花费更多计算来思考更长时间。

5.6 Goedel-Prover

哥德尔-Prover 超过 DeepSeek-Prover，金驰、陈丹琦团队造出当前最强形式化推理模型

Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving

https://github.com/Goedel-LM/Goedel-Prover

5.7 Kimi K1.5

追平满血版 o1 的国产多模态模型终于来了！训练细节全部公开

https://github.com/MoonshotAI/kimi-k1.5

Kimi k1.5: Scaling Reinforcement Learning with LLMs，自己转存了一份：pdf

5.8 O1 Embedder

O1 Embedder: Let Retrievers Think Before Action

生成关于输入查询的 thought，然后和 question 一起拼接，然后分别独自生成嵌入，然后池化聚合。也就是说，这个 Embedding 模型比之前的模型多了个 thought 的输出。

微调，两个并行任务，一个是 thought 生成，一个是对比学习。

微调数据怎么来，那就是生成后进行打分评价。首先，使用 LLM 生成初始思想，然后使用检索评分器来根据初始思想和目标文档之间的相关性评分，最终通过多数投票选择最佳 thought。

5.9 bridge

From Few to Many: Self-Improving Many-Shot Reasoners Through Iterative Optimization and Generation

问题背景：传统的 LLM 在处理长文本时效率低下，尤其是在需要进行多步推理的任务中。近年来，随着长上下文 LLM 的发展，出现了多示例学习（many-shot ICL）的范式，即通过更多的示例来提升模型性能。然而，这种范式的效果和影响因素仍需进一步研究。

研究动机：为了提升多示例学习的效率，分析了影响多示例学习效果的关键因素，并发现**仅少数关键示例**就能显著提升性能。基于这一发现，团队提出了一种新的算法，通过**优化选择关键示例**和**生成新的示例**来进一步提升模型的推理能力。

bridge 算法结合了优化和生成两个步骤：

- 在优化步骤中，使用贝叶斯优化（Bayesian optimization）来发现关键示例；
- 在生成步骤中，利用这些关键示例作为示范，重新生成更多的示例，从而在多示例学习中提高模型性能。

5.10 BFS-Prover

超越 DeepSeek-ProverV1.5！豆包首个形式化数学推理模型 BFS-Prover 来了，直接开源

BFS-Prover: Scalable Best-First Tree Search for LLM-based Automatic Theorem Proving

https://huggingface.co/bytedance-research/BFS-Prover/tree/main

5.11 CoE

为 DeepSeek MoE 模型带来「免费午餐」加速，专家链可大幅提升 LLM 的信息处理能力

5.12 显式 CoT

揭示显式 CoT 训练机制：思维链如何增强推理泛化能力

Unveiling the Mechanisms of Explicit CoT Training: How Chain-of-Thought Enhances Reasoning Generalization

5.13 认知行为

为什么 Qwen 能自我改进推理，Llama 却不行？斯坦福找到了原理

Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs

5.14 DR, GRPO

揭秘 DeepSeek R1-Zero 训练方式，GRPO 还有极简改进方案

Understanding R1-Zero-Like Training: A Critical Perspective

https://github.com/sail-sg/understand-r1-zero

5.15 AReaL

200 美金，人人可手搓 QwQ，清华、蚂蚁开源极速 RL 框架 AReaL-boba

https://github.com/inclusionAI/AReaL

5.16 TAO

模型调优无需标注数据！将 Llama 3.3 70B 直接提升到 GPT-4o 水平

https://www.databricks.com/blog/tao-using-test-time-compute-train-efficient-llms-without-labeled-data

TAO 的核心创新在于摒弃了人工标注数据，转而利用测试时计算引导模型探索任务的可能响应，再通过强化学习根据响应评估结果更新模型参数。

该流程通过可扩展的测试时计算（而非昂贵的人工标注）实现质量提升，并能灵活融入领域知识（如定制规则）。令人惊讶的是，在高质量开源模型上应用该方法时，其效果往往优于依赖人工标注的传统方案。

TAO 包含四个核心阶段：

- 响应生成：该阶段首先收集任务相关的输入提示或查询样本。在 Databricks 平台上，这些提示可通过 AI Gateway 自动采集；
- 响应评分：系统化评估生成响应的阶段。评分方法包含多种策略，例如基于奖励模型、偏好评分，或利用 LLM 评判器及定制规则进行任务特异性验证，确保每个响应都做到最优；
- 强化学习（RL）训练：最终阶段采用基于强化学习的方法更新大语言模型，引导模型生成与高分响应高度契合的输出。通过这一自适应学习过程，模型持续优化预测能力以提升质量；
- 持续改进：TAO 仅需 LLM 输入样本作为数据源。用户与 LLM 的日常交互自然形成该数据（一旦模型部署使用，即可自动生成下一轮 TAO 训练数据）。在 Databricks 平台上，借助 TAO 机制，模型会随着使用频次增加而持续进化。

虽然 TAO 在训练阶段使用了测试时计算，但最终产出的模型在**执行任务时仍保持低推理成本**。这意味着经过 TAO 调优的模型在推理阶段（与原版模型相比）具有完全相同的计算开销和响应速度，显著优于 o1、o3 和 R1 等依赖测试时计算的模型。实验表明：采用 TAO 训练的高效开源模型，在质量上足以比肩顶尖的商业闭源模型。

TAO 为 AI 模型调优提供了一种突破性方法：

- 不同于耗时且易出错的提示工程；
- 也区别于需要昂贵人工标注数据的传统微调；
- TAO 仅需工程师提供任务相关的典型输入样本，即可实现卓越性能。

5.17 deepcoder

DeepCoder-14B-Preview

UC 伯克利华人开源 14B「o3-mini」，代码版 R1 突袭 OpenAI 王座！

5.18 Think twice

Think Twice: Enhancing LLM Reasoning by Scaling Multi-round Test-time Thinking

多轮思考（Multi-round Thinking）：通过迭代地利用之前的答案作为提示，对模型的推理过程进行逐步优化。允许模型在多个推理轮次中重新考虑之前的答案，每次迭代只保留上一轮的最终答案，摒弃中间推理步骤。这一过程类似于人类的认知模式，有助于模型纠正常见的推理错误。

https://github.com/a-m-team/a-m-models

5.19 glm-z1

智谱深夜开源新一代 GLM 模型，推理速度快 DeepSeek-R1 八倍，还启用了全新域名 Z.ai

https://github.com/THUDM/GLM-4/blob/main/README_zh.md

https://huggingface.co/collections/THUDM/glm-4-0414-67f3cbcb34dd9d252707cb2e

5.20 d1

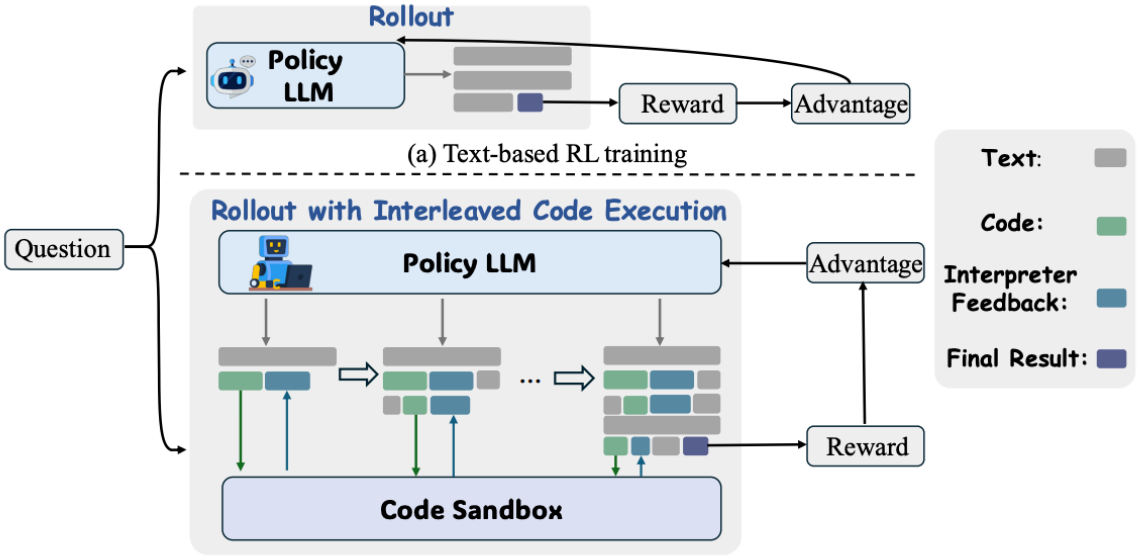
扩散 LLM 推理用上类 GRPO 强化学习！优于单独 SFT，UCLA、Meta 新框架 d1 开源

d1: Scaling Reasoning in Diffusion Large Language Models via Reinforcement Learning

https://github.com/dllm-reasoning/d1

5.21 regtool

ReTool: Reinforcement Learning for Strategic Tool Use in LLMs



(b) ReTool RL training

两阶段训练，先冷启注入 pattern，再 RL 进一步泛化

- 阶段 1：收集纯文本数学推理数据-> 人工专家和 Deepseek-R1 双重验证过滤无效数据-> 直接 prompt 模型将手动计算步骤替换为代码片段，生成代码增强的推理数据-> 格式/答案验证-> 得到的数据集用来 SFT 冷启模型
- 阶段 2：RL rollout 时候当检测到\code 时停止生成，代码片段在 sandbox 中执行，并将解释器反馈拼接到原来生成序列中，使用是否做对（+1/-1）作为奖励信号进行 PPO 训练

5.22 RLVR 的局限

RL 真让大模型更会推理？清华新研究：其能力边界或仍被基座「锁死」

Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?

RLVR（可验证奖励的强化学习）只是将采样做得更有效率，而其输出的正确答案，早已藏在基座模型的「基因」里。

5.23 RLFT

谷歌 DeepMind：大模型也很任性，知道最优路径偏要撞南墙

LLMs are Greedy Agents: Effects of RL Fine-tuning on Decision-Making Abilities

5.24 小结

从 ReFT, Kimi K1.5 到 DeepSeek R1，聊聊 Reasoning Model 的精巧实现

张俊林：MCST 树搜索会是复刻 OpenAI O1/O3 的有效方法吗

6 R1 的复现与部署

6.1 open-r1

https://github.com/huggingface/open-r1/

开源 22 万条 DeepSeek R1 的高质量数据！你也能复现 DeepSeek 了

https://huggingface.co/datasets/open-r1/OpenR1-Math-220k

6.2 unlock-deepseek

DeepSeek R1 Zero 中文复现教程来了！

6.3 open-reasoner-zero

1/30 训练步骤复刻 DeepSeek-R1-Zero，沈向洋姜大昕张祥雨等开源推理模型 RL 训练方法

https://github.com/Open-Reasoner-Zero/Open-Reasoner-Zero/

6.4 vllm 跑 gguf 的 r1

https://github.com/vllm-project/vllm/pull/13167

从源码安装参考https://docs.vllm.ai/en/latest/getting_started/installation/gpu/index.html#build-wheel-from-source

```
export https_proxy=xxxxxx
pip3 install setuptools_scm ## 不知道是不是需要的
git clone https://github.com/vllm-project/vllm.git
cd vllm
VLLM_USE_PRECOMPILED=1 pip3 install --editable .

# 如果网络有问题，可以直接 pip3 install .
```

```
# 下面这 3 步还是比较必要的
cp -r /usr/local/lib/python3.10/dist-packages/vllm /usr/local/lib/python3.10/dist-packages/vllm.bk
rm -rf /usr/local/lib/python3.10/dist-packages/vllm
cp -r ./vllm /usr/local/lib/python3.10/dist-packages
```

- 去https://huggingface.co/deepseek-ai/DeepSeek-R1/tree/main把各个非 model 的小文件下载下来（可以直接 `GIT_LFS_SKIP_SMUDGE=1 git clone https://huggingface.co/deepseek-ai/DeepSeek-R1`）也要下载，
- 去https://huggingface.co/unsloth/DeepSeek-R1-GGUF/tree/main把 config.json 下载下来，把 json 里的 torch_dtype 里的 bfloat16 改成 float16，覆盖掉上面那个目录里的 config.json

上面的那些文件都放到`./unsloth_dir` 目录下，注意，目前只能用 Q2 Q4 那种量化，1.58bit 那种动态量化不支持

```
cd ./unsloth_dir/
# merge 成一个 gguf
llama.cpp/llama-gguf-split --merge ./DeepSeek-R1-Q2_K/DeepSeek-R1-Q2_K-00001-of-00005.gguf ./unsloth_di
```

代码:

```
from vllm import LLM, SamplingParams

import multiprocessing

if __name__ == "__main__":
    # 这坨要放 main 里，deepseek 教我的
    multiprocessing.set_start_method('spawn', force=True)

    llm = LLM(model="./unsloth_dir/merge.gguf",
              tokenizer="./unsloth_dir/DeepSeek-R1",
              hf_config_path="./unsloth_dir/DeepSeek-R1",
              enforce_eager=True,
              tensor_parallel_size=8, #
              trust_remote_code=True,
              distributed_executor_backend="mp",
              max_model_len=2000)
    sampling_params = SamplingParams(temperature=0.5, max_tokens=2000)

    def print_outputs(outputs):
        for output in outputs:
            prompt = output.prompt
            generated_text = output.outputs[0].text
            print(f"Prompt: {prompt!r}, Generated text\n: {generated_text}")
        print("-" * 80)
    conversation = [
        {
            "role": "user",
            "content": " 中国的首都是哪里",
        },
    ]
    outputs = llm.chat(conversation,
                       sampling_params=sampling_params,
                       use_tqdm=False)
    print_outputs(outputs)
```

运行:

```
VLLM_MLA_DISABLE=1 VLLM_WORKER_MULTIPROC_METHOD=spawn python3 vllm_deepseek.py
```

6.5 unsloth

6.5.1 跑满血版 1.58bit infer

https://unsloth.ai/blog/deepseekr1-dynamic

6.5.2 让小模型有 reasoning 能力

https://unsloth.ai/blog/r1-reasoning

6.6 Logic-RL 与 reinforce-lite

10 美元成功复现 DeepSeek 顿悟时刻，3B 模型爆发超强推理！微软论文反驳涌现

6.6.1 reinforce-lite

https://medium.com/@rjusnba/overnight-end-to-end-rl-training-a-3b-model-on-a-grade-school-math-dataset-leads-to-reasoning-df61410c04c6

6.6.2 Logic-RL

Logic-RL: Unleashing LLM Reasoning with Rule-Based Reinforcement Learning

7 R1 的微调

7.1 ColossalAI

DeepSeek V3+R1 满血微调工具上线！一键启动，硬件要求降 10 倍

https://github.com/hpcaitech/ColossalAI

7.1.1 lora sft 满血 deepseek V3/R1

- 数据：https://github.com/hpcaitech/ColossalAI/blob/main/applications/ColossalChat/examples/training_scripts/lora_sft_data.jsonl
- fp8 权重转 bf16：https://github.com/deepseek-ai/DeepSeek-V3/blob/main/inference/fp8_cast_bf16.py
- 脚本：https://github.com/hpcaitech/ColossalAI/blob/main/applications/ColossalChat/examples/training_scripts/lora_finetune.py

通过使用 LoRA 等优化，示例命令已将 SFT DeepSeek V3/R1 671B 最低硬件要求降低近 10 倍，可使用 32 个 Ascend 910B NPU 64GB（使用 ep=8,pp=4）或 24 个 H100/H800 GPU（使用 ep=8,pp=3）。如果你通过-zero_cpu_offload 启用 CPU offload，硬件要求可以进一步降低，但会损失一定的训练速度。

```
colossalai run --hostfile path-to-host-file --nprocpernode 8 \
  lorafinetune.py --pretrained path-to-DeepSeek-R1-bf16 \
  --dataset path-to-dataset.jsonl --plugin moe \
  --lr 2e-5 --maxlength 256 -g --ep 8 --pp 3 \
  --batchsize 24 --lorarank 8 --loraalpha 16 \
  --numepochs 2 --warmupsteps 8 \
  --tensorboarddir logs --save_dir DeepSeek-R1-bf16-lora
```

也可以使用上述脚本，将并行度高效扩展至数百及数千卡，快速完成 DeepSeek V3/R1 671B 全参微调或并行加速。

7.1.2 用强化学习微调蒸馏版 DeepSeek

奖励设计：

1. 奖励 = 0，如果格式是错误的；
2. 奖励 = 1，如果格式是正确的但是结果是错误的；
3. 奖励 = 10，如果格式与结果都是正确的。

- 模板和设定：https://github.com/hpcaitech/ColossalAI/blob/main/applications/ColossalChat/conversation_template/Qwen_Qwen2.5-3B.json
- 启动脚本：https://github.com/hpcaitech/ColossalAI/blob/main/applications/ColossalChat/examples/training_scripts/train_grpo.sh

8 OpenAI

OpenAI：强化学习确实可显著提高 LLM 性能，DeepSeek R1、Kimi k1.5 发现 o1 的秘密

Competitive Programming with Large Reasoning Models

刚刚，OpenAI 放出最后大惊喜 o3，高计算模式每任务花费数千美元

9 Qwen

9.1 QwQ-max-preview

阿里旗舰推理模型硬刚 DeepSeek！官宣独立 APP，发布公告 AI 亲自写

https://chat.qwen.ai

9.2 QWQ-32B

https://qwenlm.github.io/blog/qwq-32b/

https://huggingface.co/Qwen/QwQ-32B

目前只有博客，写得挺简单的，看介绍是用了两阶段 RL：

- 1 阶段 RL：在冷启 ckpt 上，只拿代码和数学语料，基于 outcome-base reward，随着训练的进行，发现这两方面的能力都有持续提升
  - 数学：有一个 accuracy verifier，检查最终解决方案是否正确
  - 代码：有一个代码执行 server，检查代码是否通过预定义好的测试用例
- 2 阶段 RL：没说用什么语料，但用的是 general reward model 加上 rule-based verifiers 产生的 reward，发现只要少量的 step 就能提升模型的通用能力，如指令遵循、人类偏好对齐、agent 性能，而且代码和数学能力下降不明显

10 Claude

10.1 Claude 3.7 Sonnet

全球首个混合推理模型：Claude 3.7 Sonnet 来袭，真实编码力压一切对手

- 既是普通的 LLM，又是推理模型。你可以选择何时希望模型正常回答，何时希望它在回答之前思考更长时间。在标准模式下，Claude 3.7 Sonnet 是前代 Claude 3.5 Sonnet 的升级版。在扩展思维模式下，它会在回答之前进行自我反思，从而提高其在数学、物理、指令遵循、编码和许多其他任务上的表现。Anthropic 发现，两种模式下，模型的提示词工作方式类似。
- 当通过 API 使用 Claude 3.7 Sonnet 时，用户还可以控制思考预算。你可以告诉 Claude 思考不超过 N 个 token。对于任何 N 值，其输出限制为 128K 个 token。这允许用户在速度（和成本）和答案质量之间进行权衡。
- 第三，在开发自家的推理模型时，Anthropic 对数学和计算机科学竞赛问题的优化较少，而是将重点转向更能反映企业实际使用 LLM 方式的现实任务。

Claude 3.7 Sonnet System Card

11 基于 llama 的推理模型

11.1 Llama Nemotron

英伟达 253B 开源新王登场，Llama 4 三天变陪衬！直逼 DeepSeek-R1 成推理天花板

https://huggingface.co/nvidia/Llama-3_1-Nemotron-Ultra-253B-v1

12 seed-thinking

200B 参数击败满血 DeepSeek-R1，字节豆包推理模型 Seed-Thinking-v1.5 要来了

Seed-Thinking-v1.5: Advancing Superb Reasoning Models with Reinforcement Learning

12.1 DAPO

超越 DeepSeek GRPO 的关键 RL 算法，字节、清华 AIR 开源 DAPO

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

https://github.com/volcengine/verl/tree/gm-tyx/puffin/main/recipe/dapo

数据：https://huggingface.co/datasets/BytedTsinghua-SIA/DAPO-Math-17k

12.2 VAPO

字节新作 VAPO：使用基于价值的强化学习框架进行长思维链推理

VAPO: Efficient and Reliable Reinforcement Learning for Advanced Reasoning Tasks

13 多模态推理

13.1 综述

Rule-based 强化学习 ≠ 古早逻辑规则！万字拆解 o1 多模态推理最新进展

Aligning Multimodal LLM with Human Preference: A Survey

13.2 视觉推理

视觉强化微调！DeepSeek R1 技术成功迁移到多模态领域，全面开源

Visual-RFT: Visual Reinforcement Fine-Tuning

https://github.com/Liuziyu77/Visual-RFT

13.3 R1V

全球首个工业界多模态推理模型开源！38B 硬刚 DeepSeek-R1，训练秘籍全公开

https://github.com/SkyworkAI/Skywork-R1V

Skywork R1V: Pioneering Multimodal Reasoning withChain-of-Thought

13.4 gemini 2.0

OpenAI 深夜被狙，谷歌 Gemini 2.0 掀翻牌桌！最强智能体组团击毙 o1

谷歌“狙击”OpenAI，发布新一代大模型！主打 Agent+ 多模态

我扒出了 Gemini 2.0 超实时多模态幕后的黑科技，第六代 TPU 芯片 Trillium！

谷歌逆风翻盘暴击 OpenAI，90 天王者归来！44 页报告押注 25 年三大技术前沿

data and ai trends report 2024

13.5 Video-T1

视频生成的测试时 Scaling 时刻！清华开源 Video-T1，无需重新训练让性能飙升

Video-T1: Test-Time Scaling for Video Generation

https://github.com/liuff19/Video-T1

13.6 mureka o1(音乐)

音乐界迎来自己的 DeepSeek！全球首个音乐推理大模型 Mureka O1 上线，超越 Suno

MusiCoT: Analyzable Chain-of-Musical-Thought Prompting for High-Fidelity Music Generation

https://musicot.github.io/

13.7 Kimi-VL

月之暗面开源轻量级 MoE 多模态模型，支持推理，效果超过 GPT-4o！

Kimi-VL technical report

https://huggingface.co/collections/moonshotai/kimi-vl-a3b-67f67b6ac91d3b03d382dd85

Kimi 16B 胜 GPT-4o！开源视觉推理模型：MoE 架构，推理时仅激活 2.8B

14 统一的多模态模型

14.1 gemini 2.5

DeepSeek 逼出谷歌新推理模型：40 分优势超 GPT4.5 登顶竞技场，支持原生多模态，但依然败给了“竹竿问题”

谷歌终于登顶一次了！最强推理模型 Gemini 2.5 Pro 实测体验，真的有点东西

谷歌首款混合推理 Gemini 2.5 登场，成本暴降 600%！思考模式一开，直追 o4-mini

Gemini 2.5 Pro 的上下文窗口是 1M tokens，并且支持原生多模态：可以理解庞大数据集并处理来自不同信息源的复杂问题，包括文本、音频、图像、视频，甚至是整个代码库。2.5 pro 擅长创造视觉上引人注目的 Web 应用程序和智能体代码。

52 页 PPT，谷歌 Gemini 预训练负责人首次揭秘！扩展定律最优解

https://vladfeinberg.com/assets/2025-04-24-princeton-talk.pdf

14.2 gemma-3

Google 发布多模态开源模型 Gemma 3：性能与功能全面升级，附技术报告英中对照版

- 架构设计：Gemma 3 基于 decoder-only 架构，与前两代 Gemma 模型类似。但在细节上有所创新，例如采用 GQA 和 RMSNorm，还引入了 **QK-norm** 替代 Gemma 2 中的软封顶机制，以提升性能和稳定性。

- 训练策略：Gemma 3 的训练过程包括预训练和指令微调两个阶段。在预训练阶段，模型使用知识蒸馏技术，从大规模数据中学习语言和视觉表示。训练数据涵盖了文本、图像以及多语言等多种类型，且经过严格的筛选和清洗，以减少有害内容和低质量数据的影响。在指令微调阶段，Gemma 3 采用了新颖的训练方法，重点关注数学、聊天、指令遵循和多语言等能力的提升。

一台 3090 就能跑 Gemma 3 27B！谷歌发布 Gemma 3 全系 QAT 版模型

14.3 Qwen2.5-Omni

阿里深夜开源 Qwen2.5-Omni，7B 参数完成看、听、说、写

Qwen2.5-Omni Technical Report

https://github.com/QwenLM/Qwen2.5-Omni/tree/main

https://github.com/QwenLM/Qwen2.5-Omni

14.4 Qwen3

猛击 OpenAI o1、DeepSeek-R1！刚刚，阿里 Qwen3 登顶全球开源模型王座，深夜爆火

Qwen3：思深，行速

开源了两个 MoE 模型的权重：

- Qwen3-235B-A22B：拥有 2350 多亿总参数和 220 多亿激活参数
- Qwen3-30B-A3B：拥有约 300 亿总参数和 30 亿激活参数

此外，还有 6 个 Dense 模型：Qwen3-32B、Qwen3-14B、Qwen3-8B、Qwen3-4B、Qwen3-1.7B 和 Qwen3-0.6B

14.4.1 预训练

Qwen2.5 是在 18 万亿个 token 上进行预训练的，而 Qwen3 使用的数据量几乎是其两倍，达到了约 36 万亿个 token，涵盖了 119 种语言和方言。

预训练过程分为三个阶段。

- 第一阶段：模型在超过 30 万亿个 token 上进行了预训练，上下文长度为 4Ktoken。这一阶段为模型提供了基本的语言技能和通用知识。
- 第二阶段：通过增加知识密集型数据（如 STEM、编程和推理任务）的比例来改进数据集，随后模型又在额外的 5 万亿个 token 上进行了预训练。
- 第三阶段：使用高质量的长上下文数据将上下文长度扩展到 32Ktoken，确保模型能够有效地处理更长的输入。

效果：

- Qwen3 Dense 基础模型的整体性能与参数更多的 Qwen2.5 基础模型相：Qwen3-1.7B/4B/8B/14B/32B-Base 分别与 Qwen2.5-3B/7B/14B/32B/72B-Base 表现相当。特别是在 STEM、编码和推理等领域，Qwen3 Dense 基础模型的表现甚至超过了更大规模的 Qwen2.5 模型。
- Qwen3 MoE 基础模型在仅使用 10% 激活参数的情况下，达到了与 Qwen2.5 Dense 基础模型相似的性能，带来了训练和推理成本的显著节省。

14.4.2 后训练

四阶段：

- 长思维链冷启动：使用多样的的长思维链数据对模型进行了微调，涵盖了数学、代码、逻辑推理和 STEM 问题等多种任务和领域。这一过程旨在为模型配备基本的推理能力。
- 长思维链强化学习：重点是大规模强化学习，利用基于规则的奖励来增强模型的探索和钻研能力。
- 思维模式融合：在一份包括长思维链数据和常用的指令微调数据的组合数据上对模型进行微调，**将非思考模式整合到思考模型中**。确保了推理和快速响应能力的无缝结合。
- 通用强化学习：在包括指令遵循、格式遵循和 Agent 能力等在内的 20 多个通用领域的任务上应用了强化学习，以进一步增强模型的通用能力并纠正不良行为。

14.4.3 使用

vllm:

```
vllm serve Qwen/Qwen3-30B-A3B --enable-reasoning --reasoning-parser deepseek_r1
# 要禁用思考模式，可以移除--reasoning-parser 以及--enable-reasoning
```

软切换机制，可以在 enable_thinking=True 时动态控制模型的行为，即在用户 prompt 或系统 prompt 中添加/think 和/no_think 来逐轮切换模型的思考模式。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

classQwenChatbot:
    def __init__(self, model_name="Qwen3-30B-A3B/Qwen3-30B-A3B"):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(model_name)
        self.history = []

    def generate_response(self, user_input):
        messages = self.history + [{"role": "user", "content": user_input}]

        text = self.tokenizer.apply_chat_template(
            messages,
            tokenize=False,
            add_generation_prompt=True
        )

        inputs = self.tokenizer(text, return_tensors="pt")
        response_ids = self.model.generate\
          (**inputs, max_new_tokens=32768)[0][len(inputs.input_ids[0]):].tolist()
        response = self.tokenizer.decode(response_ids, skip_special_tokens=True)

        # Update history
        self.history.append({"role": "user", "content": user_input})
        self.history.append({"role": "assistant", "content": response})

        return response

# Example Usage
if __name__ == "__main__":
    chatbot = QwenChatbot()

    # First input (without /think or /no_think tags,
    # thinking mode is enabled by default)
    user_input_1 = "How many r's in strawberries?"
    print(f"User: {user_input_1}")
    response_1 = chatbot.generate_response(user_input_1)
    print(f"Bot: {response_1}")
    print("----------------------")

    # Second input with /no_think
    user_input_2 = "Then, how many r's in blueberries? /no_think"
    print(f"User: {user_input_2}")
    response_2 = chatbot.generate_response(user_input_2)
    print(f"Bot: {response_2}")
    print("----------------------")
```

```
    # Third input with /think
    user_input_3 = "Really? /think"
    print(f"User: {user_input_3}")
    response_3 = chatbot.generate_response(user_input_3)
    print(f"Bot: {response_3}")
```

MCP：可以使用 MCP 配置文件，使用 Qwen-Agent 内置的工具，或者自行集成其他工具。

```
from qwen_agent.agents import Assistant

# Define LLM
llm_cfg = {
    'model': 'Qwen3-30B-A3B',
    # Use the endpoint provided by Alibaba Model Studio:
    # 'model_type': 'qwen_dashscope',
    # 'api_key': os.getenv('DASHSCOPE_API_KEY'),
    # Use a custom endpoint compatible with OpenAI API:
    'model_server': 'http://localhost:8000/v1',  # api_base
    'api_key': 'EMPTY',
    # Other parameters:
    # 'generate_cfg': {
    #         # Add: When the response content is `<think>this is the thought</think>this is the answer
    #         # Do not add: When the response has been separated by reasoning_content and content.
    #         'thought_in_content': True,
    #     },
}

# Define Tools
tools = [
    {'mcpServers': {  # You can specify the MCP configuration file
            'time': {
                'command': 'uvx',
                'args': ['mcp-server-time', '--local-timezone=Asia/Shanghai']
            },
            "fetch": {
                "command": "uvx",
                "args": ["mcp-server-fetch"]
            }
        }
    },
  'code_interpreter',  # Built-in tools
]

# Define Agent
bot = Assistant(llm=llm_cfg, function_list=tools)

# Streaming generation
messages = [{'role': 'user', 'content':
  'https://qwenlm.github.io/blog/ Introduce the latest developments of Qwen'}]
for responses in bot.run(messages=messages):
    pass
print(responses)
```

15 deep research

15.1 local-deep-research

本地也能运行 Deep Research！支持 arXiv 平台，兼容 PDF、Markdown 等

https://github.com/LearningCircuit/local-deep-research

15.2 CycleResearcher

ICLR 2025 | 真正「Deep」的「Research」，通过强化学习实现可自主进化的科研智能体来了！

CycleResearcher: Improving Automated Research via Automated Review

https://ai-researcher.net/

https://github.com/zhu-minjun/Researcher

16 高效 reasoning

16.1 reasoning economy

港中文发布全新视角高效 Reasoning 综述！idea 已充满大脑…

Harnessing the Reasoning Economy A Survey of Efficient Reasoning for Large Language Models

https://github.com/DevoAllen/Awesome-Reasoning-Economy-Papers

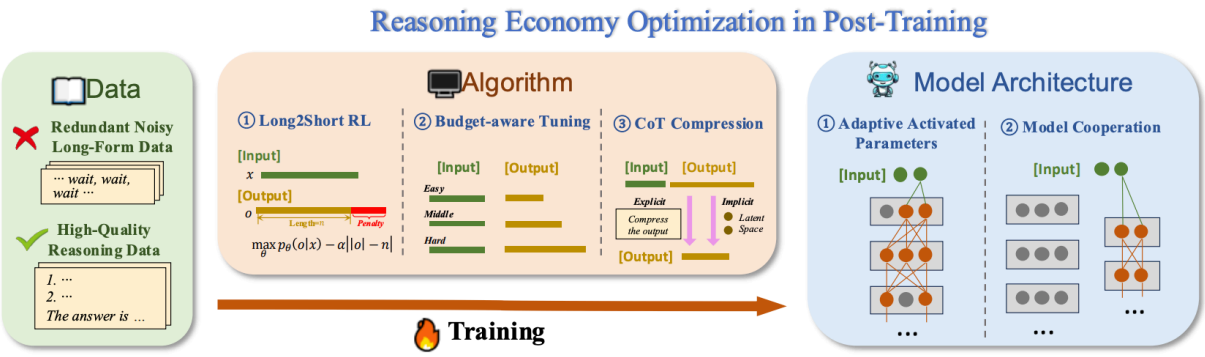


Figure 4: Post-training Methods Optimization for Reasoning Economy.

- 数据：
  - 干掉冗余、噪声、太长的数据
  - 保留高质量数据
- 算法：
  - Long2Short RL：惩罚过长的输出
  - budget-aware tuning：简单问题输出短，复杂问题输出长
  - CoT Compression：显式/隐式压缩 CoT
- 模型：
  - adaptive Activated Parameters：类似稀疏激活
  - Model Cooperation：搞 2 个模型，简单问题走简单模型，复杂问题走复杂模型

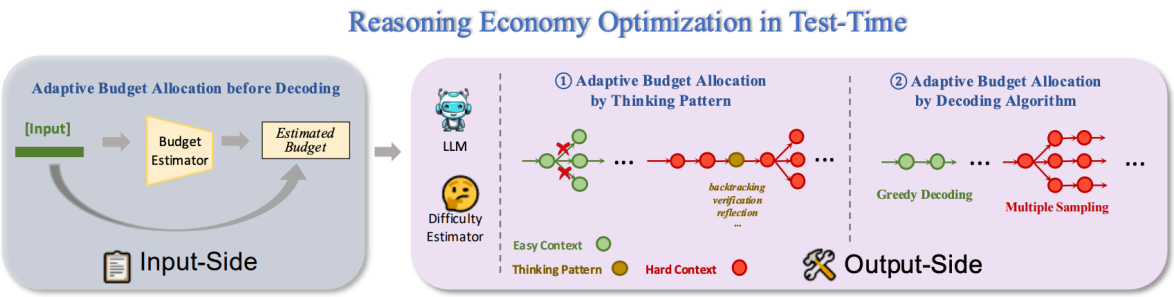


Figure 5: Test-time Methods Optimization for Reasoning Economy. The methods are divided into adding optimal computation constraint in the input-side, and selecting the best-performing decoding algorithm and controlling computation usage during decoding in the output-side.

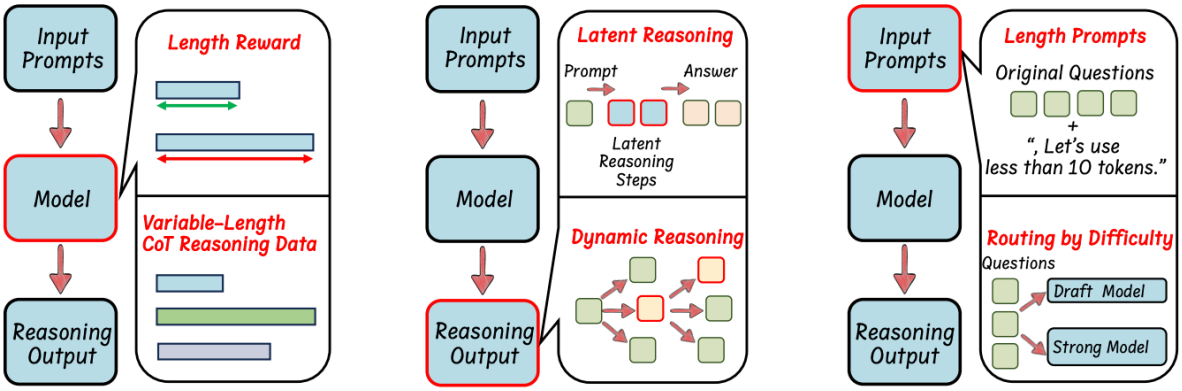
- 输入侧：解码之前进行自适应预算分配
  - 对输入进行成本预估
- 输出侧：
  - thinking pattern 的自适应预算分配：简单问题直接剪枝，复杂问题需要回溯 + 验证 + 反思
  - 解码算法的自适应预算分配：简单问题贪心解码，复杂问题多一些采样

16.2 stop overthinking

大模型还有哪些值得研究的方向？|| 大模型高效推理 (Efficient Reasoning) 研究综述

Stop Overthinking: A Survey on Efficient Reasoning for Large Language Models

https://github.com/Eclipsess/Awesome-Efficient-Reasoning-LLMs



- model：
  - length reward：RL 时加上新的 reward，鼓励答案正确且 cot 短的
  - variable-length cot reasoning data：构造不同长度 cot 的数据集，然后 sft
- reasoning-output：
  - latent reasoning：将 reasoning steps 压缩成 latent 表示
  - dynamic reasoning：reasoning 的过程改成投机采样、拒绝采样、tree-of-thoughts 等方式
- input prompt：
  - length prompts：let' s use less than k tokens
  - routing by difficulty：训一个小模型，决定简单问题不思考，困难问题再思考

17 deepseek

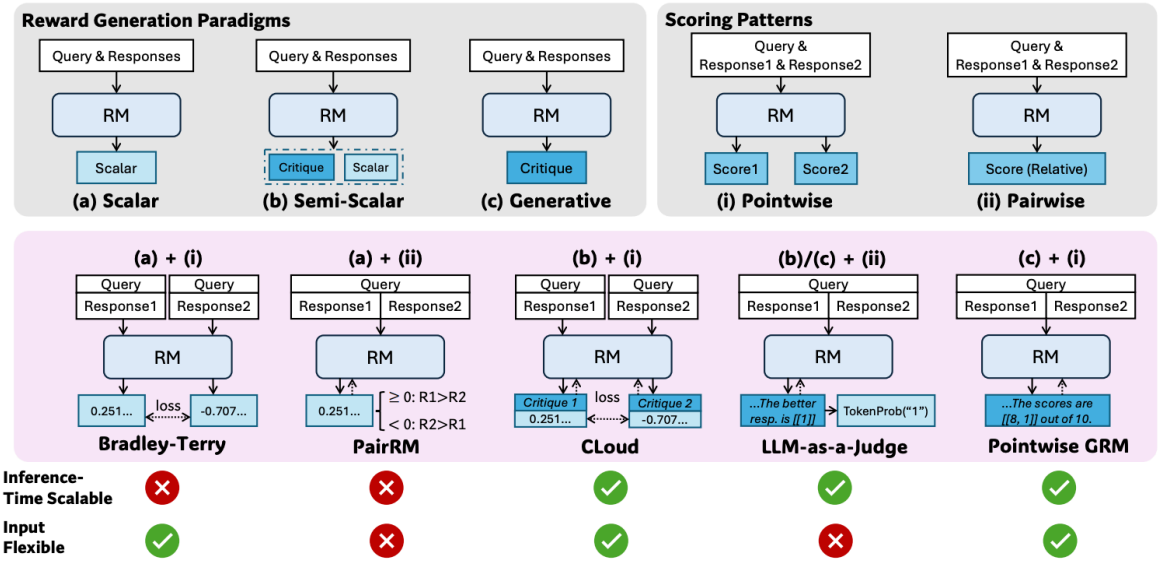
17.1 GRM+SPCT

刚刚，DeepSeek 公布推理时 Scaling 新论文，R2 要来了?

Inference-Time Scaling for Generalist Reward Modeling

critique：生成的文本评价，即评论

17.1.1 RM 分类



RM 可以进行如下分类：

- reward 生成范式：输入 query 和一些 responses
  - scalar：只输出 reward 值
  - semi-scalar：生成 critique 和 scalar
  - generative：只输出 critique
- 打分范式：输入 query 和 2 个 responses
  - pointwise：输出每个 response 的得分
  - pairwise：输出二者的相对得分

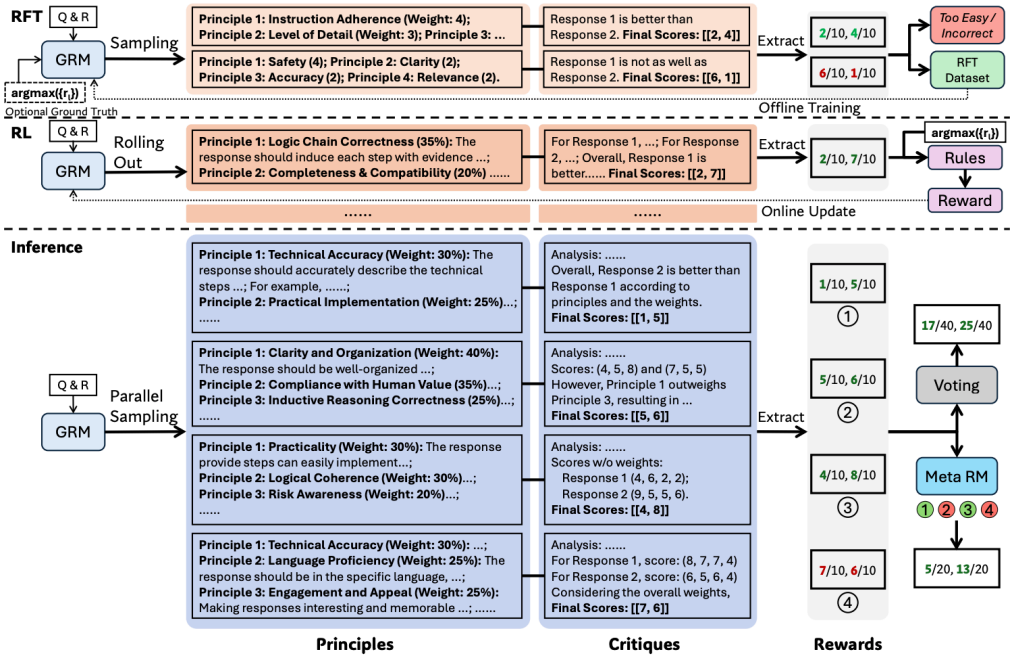
几种常见 RM：

- Bradley-Terry，scalar + pointwise
- PairRM：scalr + pairwise
- CLoud：semi-scalr + pointwise
- LLM-as-a-Judge：semi-scalr 或者 generative + pairwise
- Pointwise GRM：generative + pointwise

17.1.2 SPCT

核心：principle（准则）不再只是辅助性的输入，而是使模型能在生成过程中**主动生成**并**基于此生成对应的 critique**

Self-Principled Critique Tuning (SPCT)



- 训练（2 阶段）：输入 query+2 个 response
  - RFT（rejective fine-tuning）：使 GRM 能够生成格式正确且适配多种输入类型的 principle 与 critique
    - 使用预训练的 pointwise-GRM，采样得到 N 个 priciple + response，和 goundtruth 比较 (groundtruth 对应的 reward 是不是比其他 N-1 个结果对应的 reward 高)，满足下面条件的扔了，剩下的构造成一个数据集
      - 全对的就是 too-easy 的，扔掉；
      - 错误的也扔掉
    - 同时还有一定比例的 hint sampling，即把 groundtruth 提前泄露到 GRM 的 prompt 里去，让模型倾向于去对齐 groudtruth
  - RL（rule-based online RL）：通过不断优化生成的准则和评论，进一步增强泛化型奖励生成能力。
    - 也是用的 GRPO，只有 accuracy reward，没 format reward，加大 KL-penalty
- 推理：输入 query+2 个 response
  - 并行采样出一堆的 principles+critiques，即 m 条结果
  - voting：
    - 基于生成式 reward：即每个里的 response1 的 reward 求和，同理 response2 的 reward 求和，得到 1 和 2 的总得分
    - 基于 meta RM：训练一个 pointwise scalar RM，用二分类来判断 principle+critique 对不对 (label 和 RFT 的一样)。使用：对这 m 个结果打分，干掉质量差的（图中就是只保留了第 1 和第 3 个），然后对剩下的 k_{meta} 个去投票

17.2 DeepSeek-Prover-V2

DeepSeek 开源 Prover-V2 强推理模型，网友：奥数从没这么简单过

专为 Lean 4 打造的开源大语言模型，专注于形式化定理证明。

https://github.com/deepseek-ai/DeepSeek-Prover-V2/blob/main/DeepSeek_Prover_V2.pdf

下载了一份：pdf

两阶段训练，建立了两种互补的证明生成模式：

1. 高效非思维链（non-CoT）模式：针对快速生成正式的 Lean 证明代码进行优化，专注于生成简洁的证明，没有显式的中间推理步骤。
2. 高精度思维链（CoT）模式：系统地阐述中间推理步骤，强调透明度和逻辑进展，然后构建最终的正式证明。

与 DeepSeek-Prover-V1.5 一致，这两种生成模式由两个不同的引导提示控制。

- 第一阶段采用专家迭代，在课程学习框架内训练一个非 CoT 证明模型，同时通过基于子目标的递归证明合成难题的证明。选择非 CoT 生成模式是为了加速迭代训练和数据收集过程。
- 在此基础上，第二阶段利用了冷启动链式思维（CoT）数据，通过将 DeepSeek-V3 复杂的数学推理模式与合成形式证明相结合而生成。CoT 模式通过进一步的强化学习阶段得到增强，遵循了通常用于推理模型的标准训练流程。

DeepSeek-Prover-V2 的非 CoT 模式训练过程遵循专家迭代的范式，这是开发形式化定理证明器广泛采用的框架。在每次训练迭代中，当前最佳证明策略用于生成那些在先前迭代中未解决的难题的证明尝试。这些成功的尝试经由 Lean 证明助手验证后，被纳入 SFT 数据集以训练改进的模型。

这一迭代循环不仅确保模型能够从初始演示数据集中学习，还能提炼出自己的成功推理轨迹，逐步提高其解决更难问题的能力。总体训练过程与 DeepSeek-Prover-V1 的训练过程大致一致，仅对训练问题的分布进行了两项修改：

- Prover-V2 引入了来自自动形式化和各种开源数据集的额外问题，扩大了训练问题领域的覆盖范围。
- 新模型通过子目标分解生成的问题来扩充数据集，旨在解决 MiniF2F 基准测试有效划分中的更多挑战性实例。

在 DeepSeek-V3-Base-671B 上使用恒定的学习率 5e-6，在 16384 个 token 的上下文中进行监督微调。训练语料库由两个互补来源组成：

- 通过专家迭代收集的非 CoT 数据，生成无需中间推理步骤的 Lean 代码；
- 冷启动 CoT 数据，将 DeepSeek-V3 的高级数学推理过程提炼为结构化的证明路径。非 CoT 组件强调精益定理证明器生态系统中的形式验证技能，而 CoT 示例明确地建模了将数学直觉转化为形式证明结构的认知过程。

Prover-V2 采用 GRPO，通过为每个定理提示采样一组候选证明并根据它们的相对奖励优化策略，消除了对单独 critic 模型的需求。训练使用二元奖励，每个生成的 Lean 证明如果被验证为正确则获得 1 个奖励，否则为 0。为了确保有效学习，研究人员精心挑选训练提示，仅包括那些对监督微调模型具有足够挑战性但可解决的问题。模型在每次迭代中采样 256 个不同的问题，为每个定理生成 32 个候选证明，最大序列长度为 32768 个 token。

模型蒸馏：

- 把 DeepSeek-Prover-V1.5-Base-7B 的最大上下文长度从 4096 个 token 扩展到了 32768 个，并使用 DeepSeek-Prover-V2-671B 强化学习阶段收集的 rollout 数据对这个扩展上下文模型进行微调。
- 除了 CoT 推理模式外，还整合了专家迭代过程中收集的非 CoT 证明数据，以实现一种成本效益高的证明选项，该选项能够生成简洁的形式化输出，并且模型规模较小。
- 7B 模型也采用了与 671B 模型训练相同的强化学习阶段以提升性能。

18 推理模型总结

从自我进化视角出发，全面解析 LLM 的推理能力技术演进路径

A Survey on LLM Complex Reasoning through the Lens of Self-Evolution

https://github.com/cs-holder/Reasoning-Self-Evolution-Survey

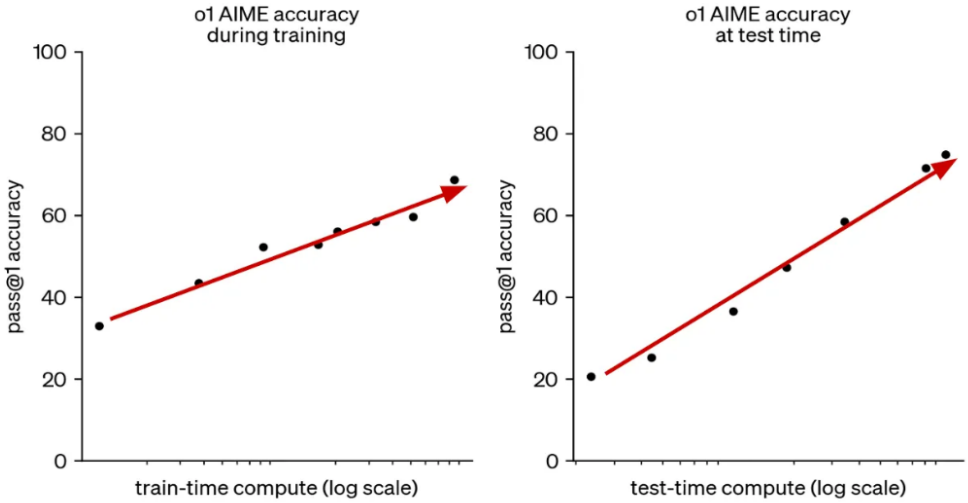
(toread)

更新的：

DeepSeek-R1 之后推理模型发展如何？Raschka 长文梳理后 R1 时代 14 篇重要论文

从 o1-mini 到 DeepSeek-R1，万字长文带你读懂推理模型的历史与技术

https://cameronrwolfe.substack.com/p/demystifying-reasoning-models

https://openai.com/index/learning-to-reason-with-llms/这里的图：说明 openai 发现了在训练时用大规模的强化学习（train-time compute）以及在测试时花更多的时间思考（test-time computing）都能提升数学任务上的效果

Power laws for compute invested into reinforcement learning and inference with o1